



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**Aplicación web para el cálculo
de trazas de herbivoría
mediante visión por
computador
Documentación Técnica**



Presentado por Marcos Zamorano Lasso
en Universidad de Burgos — 27 de mayo
de 2026

Tutor: Álgar Arnaiz González y David
Martínez Acha

Índice general

Índice general	i
Índice de figuras	iii
Índice de tablas	v
Apéndice A Plan de Proyecto Software	1
A.1. Introducción	1
A.2. Planificación temporal	2
A.3. Estudio de viabilidad	38
Apéndice B Especificación de Requisitos	45
B.1. Introducción	45
B.2. Objetivos generales	45
B.3. Catálogo de requisitos	46
B.4. Especificación de requisitos	49
B.5. Actores del sistema	50
Apéndice C Especificación de diseño	63
C.1. Introducción	63
C.2. Diseño de datos	63
C.3. Diseño procedimental	73
C.4. Diseño arquitectónico	75
C.5. Diseño web	79
Apéndice D Documentación técnica de programación	93
D.1. Introducción	93

D.2. Estructura de directorios	94
D.3. Manual del programador	96
D.4. Compilación, instalación y ejecución del proyecto	96
D.5. Pruebas del sistema	96
Apéndice E Documentación de usuario	97
E.1. Introducción	97
E.2. Requisitos de usuarios	97
E.3. Instalación	97
E.4. Manual del usuario	97
Apéndice F Anexo de sostenibilización curricular	99
F.1. Introducción	99
F.2. Relación del proyecto con la sostenibilidad	99
F.3. Competencias de sostenibilidad trabajadas	100
F.4. Decisiones de ingeniería aplicadas	100
F.5. Conclusión	101
Bibliografía	103

Índice de figuras

A.1. Burndown Report del Sprint 8	15
A.2. Burndown Report del Sprint 9	18
A.3. Burndown Report del Sprint 10	20
A.4. Burndown Report del Sprint 11	22
A.5. Burndown Report del Sprint 12	23
A.6. Burndown Report del Sprint 13	26
A.7. Burndown Report del Sprint 14	28
A.8. Burndown Report del Sprint 15	30
A.9. Burndown Report del Sprint 16	32
A.10. Burndown Report del Sprint 17	35
A.11. Burndown Report del Sprint 18	37
 B.1. Diagrama general de Casos de Uso	 51
 C.1. Diagrama entidad-relación de la base de datos	 64
C.2. Diagrama relacional de la base de datos	67
C.3. Diagrama de Secuencia de la aplicación	74
C.4. Diagrama hexagonal de la arquitectura del sistema	76
C.5. Arquitectura por capas del sistema	77
C.6. Diagrama de componentes de la aplicación	78
C.7. Diagrama de despliegue de la aplicación	79
C.8. Mockup de la web: página principal	80
C.9. Mockup de la web: imagen insertada	81
C.10. Mockup de la web: modal de trazas	82
C.11. Mockup de la web: trazas dibujadas	82
C.12. Mockup de la web: modal de error	83
C.13. Mockup de la web: menú	84
C.14. Mockup de la web: visor	84

C.15.Mockup de la web: colección de imágenes	85
C.16.Mockup de la web: galería de imágenes	86
C.17.Mockup de la web: imagen resaltada	87
C.18.Mockup de la web: registro	87
C.19.Mockup de la web: log in	88
C.20.Mockup de la web: Menú de usuarios	89
C.21.Mockup de la web: Gestión de usuarios	90
C.22.Mockup de la web: Gestión de modelos	91
C.23.Mockup de la web: Gestión de cuenta	92
D.1. Estructura principal de directorios del proyecto.	94
D.2. Estructura principal de directorios de la aplicación web.	95

Índice de tablas

A.1. Cotizaciones empresariales consideradas para el cálculo del coste de personal.	39
A.2. Estimación de costes de personal del proyecto.	39
A.3. Costes de hardware imputados al proyecto.	40
A.4. Costes de software considerados en el proyecto.	40
A.5. Estimación de costes de despliegue y mantenimiento.	41
A.6. Resumen de costes económicos del proyecto.	42
C.1. Diccionario de datos correspondiente a la tabla <i>Usuario</i>	69
C.2. Diccionario de datos correspondiente a la tabla <i>Parcela</i>	71
C.3. Diccionario de datos correspondiente a la tabla <i>Foto</i>	72
C.4. Diccionario de datos correspondiente a la tabla <i>Modelo</i>	73

Apéndice A

Plan de Proyecto Software

A.1. Introducción

La planificación de un proyecto software constituye un proceso de gestión organizado que permite coordinar actividades, recursos y plazos con el fin de garantizar una ejecución controlada y la consecución de los objetivos definidos. En el contexto de este TFG, donde se desarrolla una aplicación web para la inferencia y visualización de sendas de herbívoros a partir de imágenes aéreas, disponer de un plan de proyecto resulta especialmente relevante para estructurar el trabajo y registrar el avance de forma verificable.

Este apartado presenta, en primer lugar, la planificación temporal seguida durante el desarrollo, desglosando el proyecto en tareas e hitos y estableciendo su evolución cronológica. Para ello, se adopta un enfoque iterativo e incremental inspirado en metodologías ágiles, particularmente *Scrum*, organizando el trabajo en ciclos cortos llamados *Sprints* con los que se busca conseguir una correcta revisión del progreso y una posible adaptación ante incidencias o cambios de alcance, manteniendo una visión continua del conjunto de tareas pendientes y completadas mediante el denominado *Product Backlog*, que funciona como un registro de las labores a llevar a cabo.

Finalmente, el plan se completa con un estudio de viabilidad que analiza el proyecto desde una perspectiva económica y legal. En esta parte se consideran los costes asociados al desarrollo y despliegue, así como las implicaciones derivadas del uso de herramientas, librerías y recursos de terceros, prestando especial atención a licencias software y condiciones de

uso, con el objetivo de asegurar que la solución propuesta sea sostenible, reutilizable y coherente con un escenario de explotación real.

A.2. Planificación temporal

En este apartado se recoge la evolución del trabajo desarrollado a lo largo de los distintos *sprints*, indicando para cada uno de ellos el periodo temporal correspondiente, los objetivos planteados, la dedicación estimada y real, así como las principales tareas llevadas a cabo. Esta organización permite reflejar de forma ordenada el avance efectivo del proyecto y la manera en que se fueron cerrando, ajustando o replanificando distintos bloques de trabajo conforme la aplicación y la memoria evolucionaban.

Durante los primeros *sprints* del proyecto, la planificación y el seguimiento no se estructuraron todavía mediante una estimación homogénea en *points*, sino principalmente a partir de horas previstas y tareas registradas de forma más informal. Como consecuencia, en esta fase inicial no fue posible generar *burndown reports* consistentes ni comparables, ya que la organización del trabajo todavía no seguía un criterio de medición suficientemente estable. Por este motivo, los *burndown reports* solo se incorporan a partir del *Sprint 8*, momento en el que la planificación ya estaba mejor consolidada y permitía representar de forma más fiel la evolución de los *open points* a lo largo de cada iteración.

Sprint 1 - Documentación e investigación de conceptos teóricos

El *Sprint 1* se desarrolló del 25 de octubre al 7 de noviembre. En esta primera etapa, antes del inicio del segundo semestre, los *sprints* estaban pensados para una duración de dos semanas, y funcionó como una toma de contacto inicial con el TFG, ya que comencé a trabajar en él antes de lo establecido oficialmente: aunque la asignatura la curso en el segundo semestre, empecé a avanzar a finales del primero. Además, la semana siguiente a iniciar el TFG comencé mis prácticas en empresa y, junto con tres asignaturas en paralelo, me resultó complicado reservar horas estables para esta tarea, aunque fui sacando tiempo progresivamente.

Para este *sprint* estimé una dedicación total de 22 horas. Finalmente, dediqué aproximadamente 15 horas, lo que se reflejó en que no llegué a completar todas las *issues* previstas.

Las tareas planteadas para este *sprint* fueron:

- Documentación e investigación de conceptos teóricos asociados al proyecto (segmentación semántica, predicción densa, aprendizaje profundo, evaluación y métricas, *encoders* y modelos/arquitecturas).
- Estudio en detalle del artículo base sobre el que se apoya el TFG, para comprender su fundamentación teórica y el enfoque metodológico.
- Localización y recopilación de bibliografía para los distintos conceptos a documentar.
- Configuración del repositorio y del flujo de trabajo: GitHub, operación remota y entorno de desarrollo en VS Code.
- Puesta en marcha del entorno $\text{\LaTeX}$ con la plantilla proporcionada por los profesores y configuración de herramientas ($\text{\TeXstudio}$, $\text{\MiKTeX Console}$ y $\text{\TeXworks}$).
- Selección y configuración de la herramienta de gestión de tareas, optando por Zube.io tras descartar ZenHub.

Durante la primera semana me centré principalmente en asentar el entorno de trabajo. Configuré GitHub y el acceso para operación remota, dejé preparado VS Code para trabajar con comodidad y monté la plantilla de $\text{\LaTeX}$ proporcionada por los profesores. En paralelo, instalé y ajusté las herramientas necesarias para editar y compilar el proyecto, $\text{\TeXstudio}$, $\text{\MiKTeX Console}$ y $\text{\TeXworks}$, ya que al principio necesitaba entender bien cómo organizar el proyecto y compilar sin fricción. También exploré opciones para la gestión del tablero de trabajo: tras comprobar que ZenHub me daba un error inesperado y sin mucha posibilidad de solución, migré a Zube.io y dejé preparada la estructura para organizar las *issues*.

A lo largo de las dos semanas, la parte principal del *sprint* fue la investigación y búsqueda bibliográfica. Dediqué la mayor parte del tiempo a leer y estudiar diferentes *papers* científicos para asentar la base teórica, especialmente el artículo principal en el que se fundamenta el TFG, que trabajé en detalle para comprender correctamente su planteamiento. Como resultado de esta fase, conseguí recopilar bibliografía para la mayoría de los conceptos que tenía previstos documentar, aunque la redacción completa no llegó a cerrarse en este *sprint*.

En cuanto a la documentación escrita, el objetivo era redactar las *issues* de conceptos teóricos planificadas; sin embargo, por falta de tiempo y por la incertidumbre inicial sobre cómo estructurar el contenido, únicamente avancé

en la redacción de dos bloques: la definición de *Visión por Computador* y los conceptos teóricos generales de la *Segmentación Semántica*. Además, estos avances no quedaron consolidados mediante *commit* hasta el *sprint* siguiente, ya que durante este primer tramo estuve reorganizando la estructura del proyecto y me costó definir correctamente las bases del mismo.

Sprint 2 - Primer prototipo de la aplicación web

El *Sprint 2* abarcó del 8 de noviembre al 22 de noviembre. Para este periodo estimé una dedicación de 12,5 horas, aunque finalmente empleé 21 horas. En este punto, la planificación todavía se encontraba en una fase inicial de ajuste, por lo que el seguimiento del trabajo seguía haciéndose de forma poco estructurada y no mediante una estimación estable en *points*. En su lugar, intenté organizarme con un campo personalizado de *horas*, pero comprobé que no resultaba útil para medir progreso ni para generar informes de seguimiento.

Los objetivos principales de este *sprint* fueron los siguientes:

- Estudiar la documentación oficial de Flask y realizar los tutoriales propuestos [3].
- Diseñar el *mockup* inicial de la aplicación web.
- Implementar una primera versión sencilla de la web, con cuatro botones principales.
- Verificar el flujo de entrada/salida de la aplicación: recibir una imagen en Flask y dibujar en HTML unas líneas de esquina a esquina sobre la imagen.
- Añadir documentación del código y una primera batería de *tests* asociados a este acercamiento inicial.

Durante la primera semana me centré en dos frentes. Por un lado, diseñé el *mockup* de la web para fijar la estructura de pantallas y el flujo esperado de uso. Por otro, dediqué tiempo a estudiar Flask, siguiendo la documentación oficial y completando los tutoriales, ya que necesitaba asentar la base del *backend* antes de comenzar con una implementación funcional.

En la segunda semana abordé la implementación inicial de la aplicación. Construí un prototipo sencillo con los cuatro botones definidos y dejé operativo el flujo mínimo: la aplicación recibía una imagen en el servidor

y se mostraba en el *frontend* con un trazado simple de líneas de esquina a esquina. Este paso fue clave para validar de forma temprana el formato del JSON de trazas y confirmar que el intercambio de datos entre *backend* y *frontend* estaba funcionando como esperaba. Además, documenté el código y añadí *tests* básicos para asegurar que este acercamiento inicial quedaba mínimamente cubierto.

A lo largo del *sprint* también intenté avanzar en la redacción de conceptos teóricos arrastrados del *sprint* anterior, pero apenas conseguí progresar en esa parte debido a la carga de trabajo asociada a la puesta en marcha del prototipo. Aun así, logré completar las tareas específicas definidas para este *sprint*. La única excepción fue la integración del *mockup* en la memoria, que no pude dejar lista a tiempo por un fallo en la configuración del entorno de L^AT_EX en T_EXstudio, lo que obligó a posponer esa incorporación.

Sprint 3 - Ajustes en estilo y aplicación de la web

El *Sprint* 3 se desarrolló entre el 22 de noviembre y el 6 de diciembre. Para este *sprint* estimé inicialmente 4,5 horas, ya que en ese momento todavía no estaba creando todas las tareas de forma consistente y únicamente llegué a registrar algunas. Sin embargo, la dedicación real fue notablemente superior, alcanzando aproximadamente 18 horas, debido a que se fueron abriendo frentes adicionales de revisión y ajuste conforme avanzaba el prototipo.

Cabe destacar que, durante la reunión que dio inicio al *sprint*, conocí al que sería mi otro tutor del TFG, David Martínez, quien me orientó sobre varios aspectos relacionados con el estilo de la interfaz, como Bootstrap, Tailwind y alternativas, y sobre ciertos requerimientos esperables en la web, lo que condicionó parte de las decisiones técnicas de este periodo.

Los objetivos abordados durante este *sprint* fueron los siguientes:

- Sustituir el botón de dibujar trazas por un *checkbox*, simplificando el flujo de interacción.
- Corregir la funcionalidad asociada al PNG, de forma que la web dibujase las trazas sobre el mismo PNG original en lugar de generar un PNG nuevo.
- Investigar Bootstrap y TailwindCSS, y elaborar una comparativa entre ambos enfoques.
- Estudiar y preparar la adopción de DaisyUI como capa de componentes sobre Tailwind.

- Realizar limpieza del código, mejorar su documentación y aplicar pequeños ajustes locales para verificación de versiones.
- Continuar, de forma paralela, con la investigación teórica y la recopilación bibliográfica de conceptos arrastrados del *sprint* 1.

Durante los primeros días me centré en mejorar el prototipo existente y en cerrar cambios funcionales pequeños pero importantes. En primer lugar, sustituí el botón dedicado a dibujar trazas por un *checkbox*, ya que resultaba más coherente con el tipo de acción y evitaba pasos innecesarios en el uso. En paralelo, corregí la funcionalidad asociada al PNG: hasta ese momento, mi aproximación consistía en generar un archivo nuevo con las trazas, pero el comportamiento esperado era que la aplicación dibujase directamente sobre el mismo PNG que se estaba visualizando. Este cambio fue relevante porque simplificó el flujo de representación y alineó la salida con el objetivo de validación visual de las trazas.

Además, en esta primera semana dediqué tiempo a investigar y comparar Bootstrap y TailwindCSS, con el objetivo de entender qué enfoque se ajustaba mejor a una interfaz sencilla pero mantenible. Como resultado, dejé documentada una comparativa que me permitió aclarar las diferencias entre un *framework* basado en componentes predefinidos (Bootstrap) y uno basado en utilidades (Tailwind), teniendo en cuenta el grado de personalización, la curva de aprendizaje y el impacto en el mantenimiento del proyecto. Estos cambios quedaron reflejados junto con una primera limpieza y documentación del código, de forma que el prototipo quedase más legible de cara a iteraciones posteriores.

En la segunda semana el foco pasó a DaisyUI. Dediqué tiempo a estudiar este conjunto de componentes sobre Tailwind, explorando su filosofía y la forma de integrarlo en un proyecto web, ya que la idea era disponer de componentes consistentes sin perder la flexibilidad de Tailwind. En este proceso también realicé algunos ajustes locales para verificación de versiones y reorganicé parte del proyecto eliminando dependencias que no estaban aportando valor en ese momento, con la intención de reducir complejidad antes de iniciar una migración estética más profunda.

Por último, durante todo el *sprint* mantuve un trabajo paralelo de investigación teórica asociado a conceptos que venía arrastrando del *sprint* 1. Aunque no llegué a consolidar estos avances en *commits* específicos, continué revisando bibliografía y tomando notas sobre bloques pendientes, como, por ejemplo, aspectos relacionados con predicción densa, tipos de arquitecturas

y métricas de evaluación, con la intención de retomar su redacción cuando el entorno técnico de la web quedase más estabilizado.

Sprint 4 - Navidades

Antes de iniciar este *sprint*, conviene señalar que del 7 al 19 de diciembre no realicé trabajo enmarcado dentro de un *sprint* como tal, ya que coincidió con el periodo de exámenes y prioricé esa carga académica. En todo caso, si avancé alguna tarea, fue de forma puntual y sin una planificación cerrada.

El *Sprint 4* abarcó del 19 de diciembre al 9 de enero, coincidiendo en gran medida con el periodo de vacaciones de Navidad. Para este *sprint* estimé una dedicación total de 10 horas, aunque finalmente empleé aproximadamente 24 horas, debido a que aproveché el mayor margen de estas semanas para consolidar decisiones técnicas y recuperar trabajo arrastrado.

Los objetivos principales de este *sprint* fueron los siguientes:

- Estudiar y redactar una comparativa entre distintas *engines* posibles para la aplicación y seleccionar una opción final.
- Analizar extensiones de bases de datos para VS Code, escoger la alternativa más adecuada y documentar la decisión.
- Elaborar el diagrama de casos de uso de la aplicación y documentar los casos asociados.
- Recuperar trabajo teórico arrastrado del *sprint 1*, avanzando en la revisión bibliográfica y en la organización de conceptos.
- Crear esquemas personales y diagramas de apoyo de flujo y de secuencia para disponer de una referencia visual clara del funcionamiento interno y del estado actual del sistema.

En primer lugar, dediqué parte del *sprint* a estudiar y redactar una comparativa entre distintos *engines* que podían utilizarse en la aplicación, analizando sus ventajas e inconvenientes en términos de integración, mantenibilidad y adecuación al flujo previsto. Como resultado de este análisis, pude decantarme por una alternativa concreta y dejar justificadas las razones técnicas de la decisión, de forma que quedase trazabilidad en la memoria.

En paralelo, revisé varias extensiones de bases de datos para VS Code, ya que necesitaba una herramienta práctica para inspeccionar y gestionar el

estado de la base de datos durante el desarrollo sin fricción adicional. Tras comparar opciones, seleccioné la que mejor se ajustaba al tipo de base de datos que estaba utilizando y al flujo de trabajo diario, y dejé reflejada la comparativa en la memoria.

Respecto al diagrama de casos de uso, aunque era una tarea planificada para este *sprint*, no llegué a completarlo como entregable final. Sin embargo, avancé una parte importante del trabajo previo: identifiqué y definí los requisitos funcionales del sistema y clarifiqué el alcance de las interacciones principales. Este paso resultó especialmente útil porque me permitió dejar preparado el terreno para, en un *sprint* posterior, transformar esos requisitos en casos de uso formales y documentarlos con un formato consistente.

A pesar de estas tareas planificadas, la parte más relevante del *sprint* fue la consolidación interna del proyecto. Aproveché el periodo de vacaciones para recuperar trabajo del *sprint* 1 que todavía tenía atrasado, principalmente relacionado con la base teórica y la organización de conceptos. Además, elaboré esquemas personales sobre el funcionamiento interno de la aplicación y definí diagramas de flujo y de secuencia iniciales. Este soporte visual me permitió tener una referencia clara del estado del sistema, detectar con mayor facilidad anomalías o incoherencias en el flujo y, en general, orientar mejor las siguientes implementaciones al disponer de un recurso visual estructurado del comportamiento esperado.

Sprint 5 - Consolidación de la memoria

El *Sprint* 5 se desarrolló del 9 al 16 de enero, siendo un *sprint* de una semana. En esta etapa el foco volvió a situarse en la memoria, tanto para corregir redacciones que habían quedado poco pulidas como para recuperar parte de la carga arrastrada desde el *sprint* 1, en ese bloque de conceptos teóricos.

Para este *sprint* estimé un esfuerzo aproximado de 5 *points*. Finalmente, completé un volumen de trabajo equivalente a 18 *points*, ya que, además de los objetivos previstos, aproveché para cerrar tareas heredadas que ya estaban suficientemente encaminadas.

Los objetivos principales de este *sprint* fueron los siguientes:

- Revisar y corregir la definición de *Visión por Computador*.
- Redactar la introducción del proyecto.
- Establecer y redactar los objetivos del trabajo.

- Recuperar trabajo pendiente del *sprint* 1 y consolidar conceptos teóricos asociados a arquitecturas de segmentación semántica.

A lo largo del *sprint* realicé una revisión general de la memoria, corrigiendo diversos fragmentos que no estaban correctamente redactados y ajustando detalles de estilo para que el texto quedase más homogéneo. En particular, revisé la definición de *Visión por Computador*, reorganizando parte del contenido y refinando la terminología para que fuese más precisa y coherente con el resto del documento.

La mayor parte del tiempo la dediqué a recuperar el trabajo arrastrado del *sprint* 1. En este punto conseguí cerrar por completo el bloque de conceptos relacionado con *Arquitecturas de segmentación semántica*, que era uno de los apartados más extensos y que requería tanto redacción como consolidación de bibliografía. Este avance fue clave porque permitió dejar el capítulo de conceptos teóricos mucho más estable de cara a *sprints* posteriores.

Además, redacté la introducción del proyecto, definiendo el contexto general del TFG y el hilo conductor que seguiría el documento. En paralelo, dejé establecidos los objetivos del trabajo, aunque esta parte no quedó cerrada completamente: la estructura y el contenido principal quedaron definidos, pero el refinamiento final del texto se completaría el primer día del *sprint* siguiente.

Por último, y de forma transversal a todas las tareas anteriores, añadí y ajusté varios textos introductorios en distintas secciones para mejorar la continuidad entre apartados, facilitar la lectura y contextualizar mejor los bloques teóricos antes de entrar en definiciones más específicas.

Sprint 6 - Trabajos relacionados y preparación de integración

El *Sprint* 6 abarcó del 14 al 21 de enero, con una duración de una semana. Para este *sprint* estimé una dedicación total de 18,5 horas, y finalmente empleé 20 horas y 45 minutos.

Los objetivos definidos para este *sprint* fueron los siguientes:

- Terminar de redactar los objetivos del trabajo.
- Corregir la introducción de la memoria.
- Resolver un fallo de compilación en el documento de anexos.

- Corregir imágenes de anexos y referencias asociadas.
- Comenzar a buscar y redactar trabajos relacionados para el Capítulo 6.
- Estudiar el código real de detección de trazas y su lógica de procesado y comenzar con su integración en la aplicación.

El primer día lo dediqué a cerrar la redacción de los objetivos del trabajo, que habían quedado planteados en el *sprint* anterior. Con esto conseguí dejarlo listo para futuras revisiones menores.

La mayor parte del *sprint* la invertí en el Capítulo 6, *Trabajos relacionados*. Para ello, realicé una búsqueda activa de herramientas y *papers* científicos relacionados con el enfoque del proyecto, revisando propuestas similares y organizando la información para que fuese comparable. El capítulo quedó redactado y estructurado, aunque algunas partes se mantuvieron pendientes de completar con apoyo visual, como, alguna figura, y pequeñas aclaraciones que terminaría de cerrar en el *sprint* siguiente.

De forma paralela fui realizando arreglos puntuales en la memoria, especialmente relacionados con imágenes y referencias en el documento de anexos. En este contexto, uno de los hitos del *sprint* fue la resolución de un fallo de compilación que apareció en el L^AT_EX de anexos y que me impedía generar correctamente el PDF. Reservé un día completo para identificar el origen del error, aislar el conflicto y aplicar los cambios necesarios hasta recuperar una compilación estable del documento.

Por último, durante este *sprint* se me proporcionó el código real de detección de trazas de hervivoría que debía integrarse en la aplicación, mediante un *notebook* de entrenamiento con diversas imágenes como fuente de estudio. Aunque no llegué a comenzar su integración, sí pude dedicar tiempo a estudiarlo y analizarlo en detalle, revisando su estructura, dependencias y lógica de procesado. Este trabajo previo fue importante para entender el flujo completo y dejar preparada la base necesaria para abordar la implementación en el *sprint* siguiente con una visión más clara de los puntos críticos.

Sprint 7 - Revisión de la memoria y cumplimiento de objetivos pasados

El *Sprint 7* abarcó el periodo comprendido entre el 21 de enero y el 10 de febrero. Se trató de un intervalo especialmente amplio, en el que se concentraron cambios relevantes tanto en la memoria como en la implementación,

en consonancia con el cierre de mi periodo de prácticas y el consecuente incremento del tiempo dedicado al TFG. En conjunto, planifiqué una dedicación total de 39,5 horas para este *sprint*, aunque finalmente empleé 54 horas.

Los objetivos definidos para este *sprint* fueron los siguientes:

1. Revisar y reformular el apartado de objetivos del proyecto, con el propósito de mejorar su claridad y consistencia.
2. Refinar explicaciones del Capítulo 3, *Conceptos teóricos*, corrigiendo definiciones puntuales.
3. Reestructurar el Capítulo 6, *Trabajos relacionados*, incorporando mayor apoyo visual y completando explicaciones incompletas.
4. Redactar los bloques conceptuales pendientes correspondientes a:
 - Métricas de evaluación en segmentación semántica.
 - Tipos de *encoder*.
5. Integrar en la aplicación web la lógica del algoritmo de inferencia proporcionado.
6. Adaptar la batería de tests a los cambios introducidos.
7. Generar un fichero `requirements.txt` con las dependencias de Python del proyecto.
8. Elaborar el diagrama de casos de uso y documentar los casos asociados.

En cuanto a la revisión de los objetivos, la organicé en dos días. El primero lo dediqué a identificar las descripciones que resultaban menos precisas y a analizar memorias de años anteriores para ajustar el estilo al esperado, mientras que el segundo se centró en la reescritura y el refinamiento final del texto.

Paralelamente, corregí definiciones concretas del Capítulo 3, en particular las relativas a *Vision Transformer* y a los mecanismos residuales. Asimismo, incorporé un texto introductorio al Capítulo 6, revisé algunas redacciones y ajusté *captions* de las figuras. Finalmente, añadí una tabla comparativa entre trabajos relacionados, cuya construcción y ajuste visual realicé en dos iteraciones para asegurar su correcta visualización en el documento.

Más adelante, completé los conceptos pendientes, correspondientes a métricas de evaluación y tipos de *encoder*. La elaboración se distribuyó en cinco días: dos destinados a métricas de evaluación, primero para redactar el contenido y después para consolidar bibliografía, fórmulas y recursos visuales, y tres destinados a *encoders*, donde fue necesario realizar una revisión bibliográfica más amplia antes de cerrar la redacción.

El hito central de este *sprint* fue la integración en la aplicación web de la lógica de inferencia del *notebook* proporcionado por los tutores. Tras analizar el flujo completo, abordé una primera aproximación mediante serialización con *pickle*, la cual no conseguí llegar a implementar del todo correctamente. Por ello, opté por una solución más robusta basada en convertir los modelos por *fold* a módulos *TorchScript* autocontenidos, ya normalizados y directamente integrables en la aplicación. Posteriormente, adapté la lógica del sistema para ejecutar esta inferencia de forma funcional, cerrando esta parte el 1 de febrero.

Como consecuencia de los cambios introducidos, actualicé la batería de tests para alinearla con la nueva integración. Adicionalmente, por recomendación de uno de los tutores, generé el fichero `requirements.txt` con las versiones de las dependencias utilizadas hasta el momento, al disponer ya de la información necesaria.

Por último, finalicé otra tarea arrastrada de *sprints* anteriores: la creación del diagrama de casos de uso y la documentación asociada. Para ello, me apoyé en los *mockups* y en el flujo de uso previsto, lo que facilitó mantener una estructura coherente y cerrada.

Cabe destacar que, antes de la reunión que dio inicio al siguiente *sprint*, se realizó una reunión intermedia de seguimiento. Sin embargo, al considerar que todavía quedaban varios frentes abiertos y que el avance no era suficientemente cerrado, se decidió prolongar el *sprint* una semana adicional para consolidar entregables y reducir la deuda técnica acumulada.

Sprint 8 - Últimos pasos antes de la release

Este *sprint* se desarrolló del 10 al 18 de febrero, finalizando todas las tareas un día antes de lo previsto, ya que, inicialmente se contemplaba el 19 como fecha de fin. El objetivo principal era completar las últimas labores previas a la primera *release* del proyecto. No obstante, durante el avance surgieron varias dudas y fallos que condicionaban el cierre, principalmente porque el modelo que se estaba utilizando para calcular los datos de las

trazas no era el adecuado, lo que hizo necesario planificar un *sprint* adicional para consolidar esta parte.

En conjunto, estimé una dedicación total de 27 horas para este *sprint*, y finalmente empleé 28 horas y 35 minutos.

Las tareas abordadas fueron las siguientes:

- Comparativa del modelo basado en *pickle* con el de PyTorch.
- Modificación de la lógica de *upload* y *calculate* de la aplicación.
- Creación de una *cookie* de sesión con caducidad a las 24 h.
- Adaptación de la aplicación a diseño *responsive*.
- Creación de la infraestructura base para integrar Flask-Babel.
- Revisión de varios puntos de la memoria.
- Actualización del diagrama de casos de uso.
- Sustitución del CSS por DaisyUI.

En cuanto a las labores de documentación, revisé varios puntos de la memoria: añadí el nombre del alumno, el de los tutores y el título del TFG, redacté un párrafo introductorio para el Capítulo 3: *Conceptos teóricos*, corregí detalles del Capítulo 6: *Trabajos relacionados* relacionados con imágenes y redacción, y ajusté el apartado C de anexos correspondiente al diseño. En este último caso, reorganicé contenido ya que gran parte de los puntos estaban realmente vinculados al apartado 4 de la memoria: *Técnicas y Herramientas*. Aunque se trataba de muchas tareas, eran intervenciones cortas y muy localizadas.

También, dentro de los anexos, ajusté el diagrama de casos de uso para mejorar su interpretabilidad desde un punto de vista externo. En concreto, reordené la aparición de los actores, modifiqué algún caso de uso, añadí nuevas tablas para los casos incorporados y completé el campo *exception* en todas ellas para mantener la consistencia del formato.

Por otro lado, en la aplicación se llevaron a cabo varias mejoras. En primer lugar, modifiqué la lógica de las funciones de *upload* y *calculate*. Antes, al insertar una imagen, esta se subía directamente al *backend*, lo que podía provocar subidas innecesarias si el usuario seleccionaba un archivo incorrecto. Para evitarlo, implementé un flujo en el que la imagen se carga

en local y solo se envía al servidor cuando el usuario pulsa *calcular trazas*, momento en el que ya ha podido verificarla. Para ello, creé una función nueva que orquestase las dos operaciones ya existentes.

A continuación, establecí un límite de sesión de 24 h mediante *cookies*, de forma que el usuario pudiera mantener su progreso durante un margen razonable. En paralelo, trabajé en el diseño *responsive* para que, en pantallas más pequeñas, como dispositivos móviles o *tablets*, se visualizasen correctamente todos los elementos y la interfaz resultase usable en distintos dispositivos.

Más adelante, preparé la infraestructura de Flask-Babel en dos días: el primero lo dediqué a revisar la documentación y a entender su funcionamiento, y el segundo a implementar los elementos base. En esta fase instalé la biblioteca y la incorporé al `requirements.txt`, creé el archivo `Babel.cfg` para la extracción de textos, añadí un desplegable con cinco idiomas: español, inglés, francés, italiano y alemán, generé recursos visuales en formato `.svg` para las banderas asociadas y dejé definidos algunos campos de Babel, sin llegar todavía a implementar la lógica completa de traducción.

La tarea principal del *sprint* fue migrar toda la interfaz de la aplicación a DaisyUI sobre Tailwind. Este proceso lo realicé de forma progresiva durante todos los días del *sprint*, planteándome pequeños avances diarios hasta completar la migración. Primero investigué los componentes disponibles y los fui aplicando iterativamente: comencé por los botones principales: insertar imagen, calcular trazas y borrar imagen, después ajusté los modales y el sistema de *flash messaging* aprovechando los diseños predefinidos, estableciendo además que los avisos se mostrasen durante cinco segundos antes de desaparecer. Más tarde, reorganicé la zona de visualización de la imagen e incorporé una lógica para permitir el arrastre de archivos desde una carpeta externa, ya que resultaba conveniente para el flujo de uso. Durante este proceso también corregí la cabecera, añadí un *footer* con información del proyecto y del ente académico correspondiente, e incorporé una imagen de fondo suave para aportar una estética más cálida.

Finalmente, el último día realicé la comparativa entre el modelo proporcionado mediante *pickle* y el modelo en PyTorch que yo había generado. Comparé métricas de rendimiento y de salida, incluyendo tiempos de carga desde disco, tiempo medio de inferencia y desviación estándar, *IoU* y *Dice* entre predicciones, número de píxeles clasificados como positivos, longitud del esqueleto en píxeles, cantidad de componentes conexas y grosor medio estimado, entre otros. Tras el análisis, concluí que el modelo en PyTorch perdía demasiada capacidad lógica frente al margen de mejora temporal

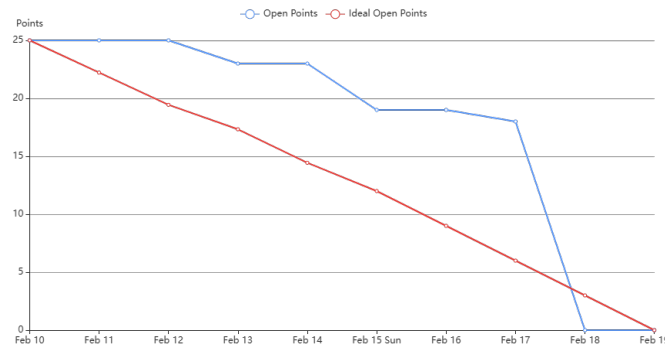


Figura A.1: Burndown Report del Sprint 8.

obtenido, por lo que resultaba conveniente dedicar un *sprint* adicional antes del cierre de la *release* para dejar esta lógica correctamente integrada.

Como se observa en el *burndown* (Figura A.1), la reducción de *open points* fue más lenta que la ideal durante la mayor parte del *sprint*, debido a los ajustes iterativos en la interfaz y la infraestructura, tareas que ocupaban un gran número de puntos, aunque fueron siendo ejecutadas durante toda la semana; no obstante, en los últimos días se cerró un bloque grande de trabajo, quedando el *sprint* prácticamente completado el 18 de febrero, un día antes de lo planificado.

Sprint 9 - Cierre de la release

Este *sprint* abarcó del 19 al 26 de febrero, con una duración de una semana. La planificación inicial contemplaba 22 horas de trabajo, a las que se sumaban 2 *points* de una tarea erróneamente realizada del *sprint* anterior, por lo que la carga prevista equivalía aproximadamente a 24 horas. Finalmente, la dedicación real fue de 25 horas y 25 minutos.

Los objetivos definidos para este *sprint* fueron los siguientes:

- Volver a ajustar el diagrama de casos de uso.
- Finalizar la lógica de Babel y completar la traducción de la aplicación.
- Añadir referencias a imágenes y corregir varios detalles del Capítulo 6, *Trabajos relacionados*.
- Redactar la comparativa entre los modelos basados en *pickle* y PyTorch.

- Corregir distintos aspectos del código y de la interfaz de la aplicación:
 - Eliminar la duplicidad existente en la ruta `calculate`.
 - Extraer el *script* embebido en `index.html` a un fichero `.js` independiente.
 - Crear un hipervínculo al sitio web de la UBU en el *footer*.
 - Eliminar el botón específico de trazas.
 - Sustituir las alertas previas por notificaciones flotantes tipo *toast*.
 - Crear e insertar un logotipo para la web (*favicon*).
 - Corregir el comportamiento visual del campo *choose file*.
- Sustituir la lógica de inferencia basada en PyTorch por una nueva lógica adaptada a los modelos en *pickle*.
- Implementar la descarga de la imagen original, la imagen resultante y el JSON de trazas.
- Crear una tabla descriptiva de *encoders* y corregir varios aspectos del Capítulo 3, *Conceptos teóricos*.
- Completar la redacción de *sprints* anteriores que todavía tenía pendientes.

En primer lugar, volví a ajustar el diagrama de casos de uso, ya que todavía no estaba completamente cerrado a nivel visual. Para ello, reorganicé la disposición de la mayor parte de los casos de uso y afiné su colocación para que el resultado fuese más claro y equilibrado desde el punto de vista gráfico.

A continuación, terminé la lógica de traducción mediante Flask-Babel. Como en el *sprint* anterior me había limitado a dejar preparada la infraestructura, en este ya encapsulé todos los textos necesarios y completé su traducción a los otros cuatro idiomas disponibles: inglés, francés, italiano y alemán, utilizando los correspondientes ficheros `.po`. Esta parte la desarrollé en dos días distintos, dejando así cerrada la base de internacionalización de la aplicación.

Después, ajusté algunos elementos del apartado de *Trabajos relacionados* que todavía no estaban del todo pulidos y redacté la comparativa entre los modelos en *pickle* y en PyTorch a partir de los resultados obtenidos en el *notebook* de pruebas del *sprint* anterior. Con ello, dejé documentada la justificación técnica del cambio de enfoque en la inferencia.

Posteriormente, abordé varios cambios en el código y en la interfaz de la web. En esta fase eliminé la duplicidad de la ruta `calculate`, separé el código JavaScript que se encontraba embebido en `index.html` para pasarlo a un fichero `.js` independiente y dejé la estructura del *frontend* algo más limpia y mantenible. También añadí un hipervínculo al sitio web de la UBU en el *footer*, eliminé el botón específico de visualización del `.json` de trazas al no resultar ya necesario dentro del flujo definitivo, sustituí las alertas previas por notificaciones flotantes tipo *toast*, incorporé un *favicon* propio para dar una identidad visual más cuidada a la aplicación y corregí el comportamiento del campo de selección de archivos, que visualmente no estaba mostrando un resultado adecuado y no se podía traducir correctamente con Flask-Babel.

Más adelante, llevé a cabo la tarea central del *sprint*, que fue sustituir toda la lógica de inferencia que había preparado para PyTorch por una nueva lógica adaptada a los modelos serializados mediante *pickle*. Esta transición me ocupó un par de días, ya que requería reajustar el flujo de carga y ejecución para que el comportamiento en la aplicación quedase alineado con la decisión técnica tomada tras la comparativa previa.

Además, implementé la lógica de descarga tanto de la imagen original como de la imagen resultante y del JSON de trazas, con el objetivo de que el usuario pudiese conservar fácilmente los archivos generados durante el procesamiento. Con ello, la aplicación resultó más completa desde el punto de vista funcional y más cercana al cierre de la primera *release*.

Por último, realicé varias correcciones en el Capítulo 3 del documento de la memoria, destacando especialmente la creación de una tabla para presentar de forma estructurada los *encoders* empleados en el artículo científico en el que baso el proyecto [2]. Paralelamente, aproveché este *sprint* para terminar de redactar los *sprints* anteriores que seguían pendientes desde hacía varias iteraciones, dejando así la planificación temporal mucho más alineada con el estado real del trabajo.

Como se observa en el burndown de este *sprint* (Figura A.2), durante los primeros días el avance fue más lento e incluso hubo un ligero repunte de *open points*, debido a que seguían apareciendo ajustes derivados del cierre técnico de la *release*. No obstante, a partir de la resolución de las tareas principales de integración y refactorización, el volumen pendiente descendió con rapidez, quedando el *sprint* completamente cerrado el 26 de febrero, cerrando así la primera *release* ese mismo día tras la reunión semanal.

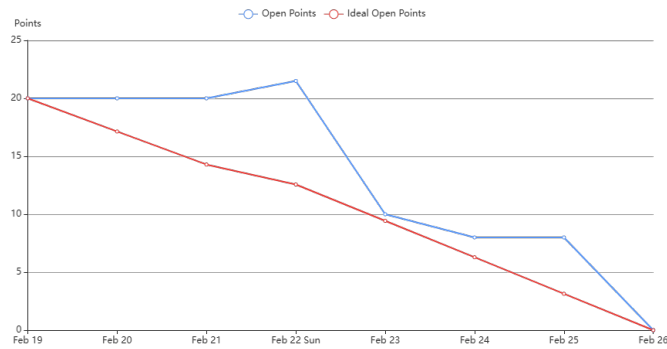


Figura A.2: Burndown Report del Sprint 9.

Sprint 10 - Primeros pasos con Google Earth Engine

El *Sprint 10* se centró en abrir una nueva línea de trabajo orientada a la integración de Google Earth Engine (GEE) como alternativa para la obtención de imágenes de entrada, a la vez que se consolidaban pequeños ajustes en la aplicación web y en la documentación. Este *sprint* abarcó del 26/02/26 hasta el 05/03/26 y estimé una dedicación total de 25 horas, aunque finalmente invertí 21,4 horas, al poder cerrar varias tareas con menor esfuerzo del previsto.

Los objetivos definidos para este *sprint* fueron los siguientes:

- Aplicar cambios puntuales en la aplicación web:
 - Renombrar el *checkbox* para mostrarlo como: Dibujar trazas.
 - Permitir que un clic sobre *upload image* activase también la selección de archivo.
- Generar el listado de dependencias mediante `pip freeze`.
- Eliminar el reescalado de imagen en una prueba concreta para verificar correctamente el dibujado de trazas.
- Corregir algunos puntos de la memoria y anexos:
 - Ajustar el formato de listados asociados a métricas como TP, FP, FN, etc.
 - Explicar la incidencia e interpretación de los gráficos *burndown* en el apartado A de anexos.

- Configurar extensiones y convenciones PEP8 y reforzar la documentación del código.
- Arreglar y consolidar *tests* de flujo.
- Iniciar las primeras implementaciones con Google Earth Engine:
 - Investigar licencias y límites asociados a la nueva implementación.
 - Crear un *notebook* base de GEE (autenticación, consulta inicial y *preview*).
 - Preparar un *notebook* para descarga de imágenes mediante colecciones de GEE.
 - Construir un prototipo HTML mínimo para seleccionar un área mediante dos clics y descargar imágenes del rectángulo definido para su posterior segmentación.

En primer lugar, realicé varios ajustes pequeños pero necesarios en la aplicación web para mejorar la claridad del flujo de uso. En concreto, renombré el *checkbox* asociado a la visualización para que apareciese como Dibujar trazas, alineándolo con la lógica del momento de la interfaz, y adapté el comportamiento del botón de *upload image* para que un clic sobre el propio elemento activase también la selección del archivo, haciendo la interacción más directa.

A continuación, consolidé el entorno de desarrollo y documentación del proyecto. Por un lado, generé el listado de dependencias mediante `pip freeze` para dejar registradas las versiones exactas del entorno. Por otro, configuré extensiones de estilo PEP8 y reforcé la documentación del código, de forma que la base quedase más mantenible y homogénea. En paralelo, realicé una prueba concreta eliminando el reescalado de la imagen, ya que necesitaba verificar el flujo del funcionamiento del dibujado de trazas. Tras esto, logré conseguir una versión correcta del mismo.

También dediqué parte del *sprint* a corregir detalles en la memoria y anexos.

Hacia el final del *sprint* incorporé una tarea no prevista inicialmente: arreglar y consolidar *tests* de flujo. Esta necesidad surgió el martes 03/03/26, a dos días del cierre del *sprint*, y me sirvió para asegurar que los cambios recientes no rompían el comportamiento esperado en el circuito principal de uso.

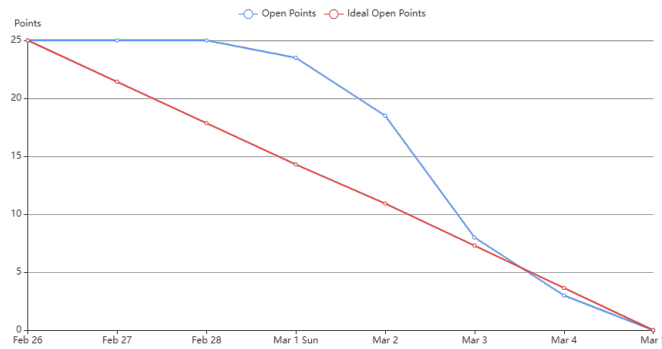


Figura A.3: Burndown Report del Sprint 10.

Por último, el bloque más relevante del *sprint* fue el inicio de las primeras implementaciones con Google Earth Engine. Comencé investigando licencias y límites de uso para entender las restricciones reales del servicio. Aquí encontré el que probablemente sea el mayor problema del TFG y es que existían limitaciones legales y funcionales para poder llevar a cabo el flujo completo de la aplicación.

A partir de ahí, construí un *notebook* base que resolvía autenticación, una primera consulta y un *preview* de resultados y preparé un segundo, tercer y cuarto *notebook* orientados a la descarga de imágenes mediante colecciones de GEE.

Más adelante, desarrollé un prototipo HTML mínimo en el que el usuario podía seleccionar un área geográfica mediante dos clics, definiendo así un rectángulo y, a partir de esa selección, descargar imágenes del recuadro para su posterior segmentación. Este avance dejó preparada la base conceptual y técnica para continuar con una integración más completa en los siguientes *sprints*.

Como se aprecia en el *burndown* del *sprint* (Figura A.3), el volumen de *open points* se mantuvo prácticamente estable durante los primeros días, ya que el trabajo se concentró en investigación y preparación y en tareas de ajuste poco visibles. A partir del 2 de marzo el descenso se aceleró al cerrarse los hitos principales y consolidarse los cambios, quedando el *sprint* completamente finalizado el 5 de marzo.

Sprint 11 - Diseños para la nueva funcionalidad

El *Sprint 11* se desarrolló entre el 5 y el 12 de marzo. Para este *sprint* estimé una dedicación total de 13 horas, y finalmente invertí 12 horas y 35 minutos. En este caso, la planificación ya contemplaba que dispondría de poco tiempo durante la semana, por lo que la mayor parte del trabajo se concentró en los últimos días del *sprint*, avanzando únicamente en tareas clave y de preparación.

Los objetivos definidos para este *sprint* fueron los siguientes:

- Corregir el *label* asociado al estado de la imagen en la aplicación.
- Mejorar algunos *tests* para que provocasen errores de forma controlada y validasen correctamente el comportamiento esperado.
- Crear el *mockup* de las nuevas pantallas necesarias para la siguiente funcionalidad de la aplicación.
- Diseñar el modelo de datos: elaborar el diagrama E-R de la aplicación.

En primer lugar, realicé un ajuste puntual en la interfaz corrigiendo el *label* que indicaba el estado de la imagen, ya que en algunos casos no reflejaba correctamente la situación del flujo. En paralelo, reforcé ciertos *tests* para que, en escenarios concretos, generasen fallos controlados y permitiesen comprobar que el sistema respondía de forma adecuada ante entradas no válidas o estados inconsistentes.

El bloque principal del *sprint* consistió en preparar el diseño de la nueva funcionalidad. Por un lado, elaboré el *mockup* de las pantallas adicionales para fijar el flujo de interacción y clarificar qué elementos debían incorporarse en la aplicación. Por otro, trabajé en el diseño de base de datos: mi intención era construir el diagrama entidad-relación (E-R), pero terminé generando directamente el esquema relacional. Posteriormente detecté que esta aproximación no era la correcta para el entregable esperado, por lo que dejé previsto corregirlo y completar el diagrama en el *sprint* siguiente.

Finalmente, al cierre del *sprint*, subí al repositorio los *notebooks* desarrollados en el *sprint* anterior. No tenía claro si debían incluirse como parte del código versionado o mantenerse únicamente como material de trabajo, y opté por incorporarlos para asegurar trazabilidad, aunque lo hice con una iteración de retraso respecto a cuando se habían elaborado.

Como se aprecia en la Figura A.4, el avance se mantuvo prácticamente plano durante la mayor parte del *sprint*, concentrándose el cierre de tareas

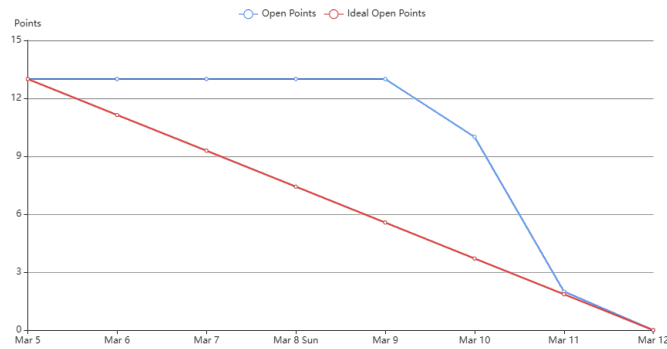


Figura A.4: Burndown Report del Sprint 11.

en los últimos días, en línea con la disponibilidad reducida que se había previsto para esta semana.

Sprint 12 - Implementación de la funcionalidad geoespacial

El *Sprint 12* se desarrolló del 12 al 19 de marzo. Para este *sprint* estimé una dedicación total de 17,5 horas y finalmente invertí aproximadamente 18 horas. El objetivo principal de esta iteración fue empezar a materializar la nueva funcionalidad geoespacial en la aplicación, preparando la pantalla referente al visor web y parte de la navegación necesaria para llegar a él.

Los objetivos definidos para este *sprint* fueron los siguientes:

- Arreglar el diagrama relacional de la base de datos.
- Crear el diagrama entidad-relación (E-R) de la BBDD.
- Investigar el alcance del PNOA y su adecuación como fuente de datos.
- Crear un menú de navegación en la *navbar*.
- Adaptar el prototipo inicial al visor web.
- Arreglar el apéndice B de los anexos.
- (Pendiente) Crear dos pantallas nuevas asociadas a la funcionalidad geoespacial: colección y galería de imágenes.

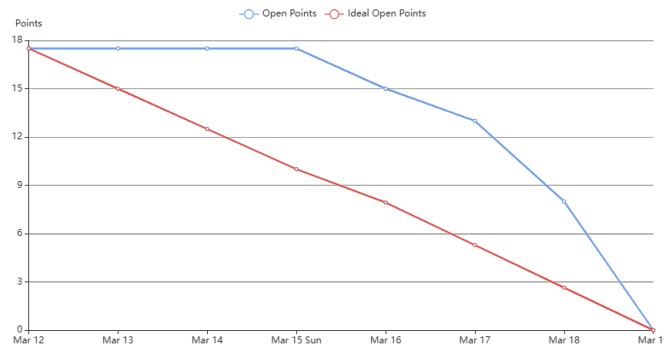


Figura A.5: Burndown Report del Sprint 12.

Durante los primeros días me centré en consolidar la estructura de la base de datos, ya que era imprescindible para que la nueva funcionalidad quedase bien definida. En primer lugar, corregí el esquema relacional que había generado en el *sprint* anterior, y a continuación elaboré el diagrama E-R de la aplicación, de manera que el diseño quedase documentado de forma correcta y consistente antes de seguir ampliando la implementación.

En paralelo, dediqué tiempo a investigar el alcance del PNOA, revisando qué cobertura ofrecía y en qué condiciones podía utilizarse dentro del flujo previsto para el proyecto. Esta parte fue relevante para validar que la fuente de datos era viable y para acotar el tipo de consultas y descargas que podrían integrarse más adelante.

En cuanto a la aplicación, implementé un menú en la *navbar* para preparar la navegación entre pantallas, y adapté el prototipo previo para que encajase con el visor web, dejando un flujo mínimo funcional y alineado con el enfoque geoespacial. Sin embargo, aunque dentro de la planificación se contemplaban también dos pantallas adicionales, que suponían aproximadamente 8 horas de trabajo y ya estaban contabilizadas dentro de las horas estimadas, no llegué a completarlas en este *sprint*. La mayor parte del esfuerzo se concentró en estabilizar la base y en dejar el visor preparado, posponiendo esas pantallas para la siguiente iteración.

Además, el lunes 16 surgió una tarea no prevista inicialmente: corregir el apéndice B de los anexos. La cerré ese mismo día, aprovechando que se trataba de un ajuste acotado y necesario para mantener la coherencia del documento.

Como se aprecia en la Figura A.5, el *burndown* se mantiene prácticamente plano durante los primeros días, reflejando que gran parte del trabajo estuvo

orientado a preparación e investigación y a tareas que no cerraban *points* de forma inmediata. A partir de la mitad del *sprint* el descenso se vuelve más pronunciado, coincidiendo con el cierre progresivo de tareas técnicas y con la incorporación y resolución de la incidencia del apéndice B. Finalmente, el *sprint* se completa el 19 de marzo, quedando únicamente como arrastre la creación de las pantallas de colección y galería, previstas pero no finalizadas.

Sprint 13 - Colecciones de imágenes

El *Sprint 13* se desarrolló del 20 al 26 de marzo. Para este *sprint* estimé una dedicación de 19 horas, y finalmente invertí 20 horas y 35 minutos. El objetivo principal de esta iteración fue completar la funcionalidad asociada a la gestión de colecciones y galerías de imágenes, cerrando además ajustes pendientes en la documentación y realizando una nueva validación del flujo de descarga con PNOA.

Los objetivos definidos para este *sprint* fueron los siguientes:

- Arreglar la bibliografía de anexos.
- Corregir y consolidar los diagramas E-R y relacional:
 - Eliminar claves foráneas del diagrama E-R para mantenerlo en un nivel de abstracción correcto.
 - Ajustar la longitud de ciertos campos.
 - Añadir un nuevo campo booleano para indicar si una imagen tiene las trazas calculadas o no.
- Arreglar el *checkbox* e incorporar la funcionalidad nueva asociada.
- Actualizar los *mockups* para incorporar cambios derivados de la nueva funcionalidad:
 - Añadir un indicador por imagen en la galería para mostrar si ya tiene las trazas calculadas.
 - Incluir, en la visualización ampliada, opciones para dibujar y calcular trazas.
 - Reflejar el estado de cada colección, apoyándolo en consultas a base de datos.
 - Incorporar opciones de eliminación de elementos en la colección.
 - Permitir la descarga de un *.zip* desde la colección.

- Ajustar el formato de las imágenes a apaisado, manteniendo la relación 1024×640 .
- Realizar pruebas con PNOA:
 - Cambiar el nombre del *manifest* a `log.json`.
 - Probar de nuevo la descarga y verificar el fallo de conexión por *time limit*.
- Revisar y ajustar el material gráfico de la memoria:
 - Exportar diagramas a PDF.
 - Actualizar el *mockup* para incluir todas las páginas y redactar una breve explicación para cada una.
- Crear las pantallas correspondientes a la nueva funcionalidad de colección y galería de imágenes.

En primer lugar, realicé ajustes, centrados en corregir la bibliografía correspondiente a los anexos. En paralelo, consolidé el diseño de base de datos: revisé tanto el diagrama E-R como el relacional, eliminando del E-R las claves foráneas para que quedase correctamente expresado a nivel conceptual, ajustando la longitud de algunos campos de texto y añadiendo un nuevo atributo booleano en la tabla de imágenes para reflejar de manera directa si las trazas estaban calculadas o no.

A continuación, corregí el *checkbox* de la aplicación para aportarle una nueva funcionalidad. Actualicé también los *mockups*, incorporando indicadores de estado por imagen, opciones en la vista ampliada para dibujar y calcular, y el estado general de cada colección. Además, añadí acciones complementarias, siendo estas la eliminación de elementos dentro de una colección y la descarga de un `.zip` directamente desde esa pantalla.

En el apartado visual, ajusté el formato de las imágenes, obtenidas mediante el mapa, a apaisado manteniendo la relación 1024×640 , de forma que se pudiese acercar más a aquellas que se usaron para entrenar el modelo. También actualicé el material gráfico de la memoria: exporté diagramas a PDF y reorganicé el *mockup* para incluir todas las páginas, añadiendo una descripción para todas y cada una de ellas.

Por otro lado, realicé ciertas pruebas de conexión con PNOA y cambié el nombre del *manifest* a `log.json`.

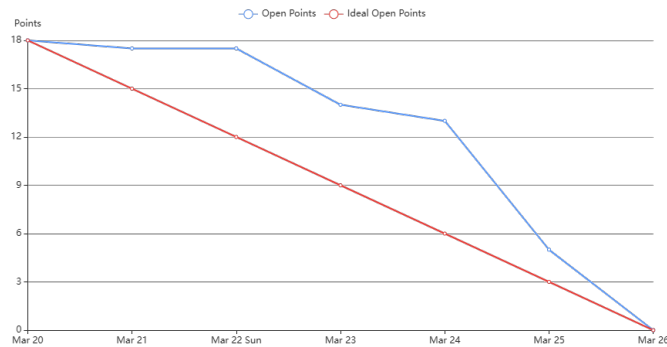


Figura A.6: Burndown Report del Sprint 13.

Finalmente, el bloque central del *sprint* consistió en crear las diferentes pantallas asociadas a la funcionalidad: la pantalla de colección y la galería de imágenes. Para llevar esto a cabo, tuve que definir también la base de datos con SQLite, como he comentado antes y preparar un lanzado automático de la misma.

Como muestra la Figura A.6, el avance se mantuvo relativamente estable durante los primeros días, al concentrarse el trabajo en ajustes de diseño y en tareas de preparación. A partir del 23–24 de marzo el descenso se acelera, coincidiendo con el cierre de las pantallas principales y la consolidación de la nueva lógica de estado en la interfaz, quedando el *sprint* cerrado el 26 de marzo.

Sprint 14 - Ajustes y consolidación durante Semana Santa

El *Sprint 14* se desarrolló durante el periodo de Semana Santa. Para esta iteración estimé una dedicación total de 26 horas, y finalmente invertí 24 horas y 25 minutos. En esta iteración el avance estuvo condicionado por carga académica: durante los primeros días apenas pude progresar y, además, el *sprint* se amplió un día adicional. A partir de ese punto, el grueso del trabajo se concentró en la segunda mitad del *sprint*, donde fui cerrando tareas de forma progresiva hasta completar el bloque planificado.

Los objetivos definidos para este *sprint* fueron los siguientes:

- Modificar el apartado de métricas en la memoria.
- Ajustar el *layout* del mapa.

- Modificar la lógica de imágenes para mantener la relación de aspecto.
- Incorporar *tooltips* en el visor.
- Añadir un campo de descripción para cada zona.
- Cambiar la previsualización de imagen por la previsualización de la zona seleccionada.
- Implementar la lógica de trazas en segundo plano.
- Redactar una explicación del problema de licencias asociado a GEE.
- Corregir aspectos relevantes de la memoria.
- Implementar la lógica de visualización de imagen en grande.
- Investigar posibles errores en el mapa.
- Investigar posibles errores en la colección de imágenes.

Durante los primeros días prioricé ajustes de documentación y cambios que no requerían una implementación extensa. En concreto, revisé el apartado de métricas de la memoria para mejorar su claridad, y realicé correcciones puntuales en secciones relevantes del documento, con el objetivo de mantener el texto alineado con la evolución real de la aplicación.

A continuación, centré el trabajo en consolidar el flujo del visor geo-espacial. Por un lado, ajusté el *layout* del mapa e incorporé *tooltips* para mejorar la interpretabilidad de la interfaz. Por otro, modifiqué la lógica de imágenes para conservar correctamente la relación de aspecto, para que así las imágenes se parecieran más a las de entrenamiento. Hice también que se pudiera cambiar el nombre de cada galería y cambié la previsualización para que, en lugar de mostrar únicamente la primera imagen, se mostrase la *preview* de la zona seleccionada completa.

En la parte más técnica del *sprint*, creé la lógica de trazas en segundo plano y la visualización de la imagen en grande, de forma que la aplicación pudiese gestionar mejor operaciones de cálculo sin bloquear la experiencia de uso y, al mismo tiempo, permitir una inspección más cómoda del resultado. Además, documenté de forma específica el problema de licencias asociado a GEE, ya que era necesario dejar constancia en la memoria de las limitaciones y condicionantes de uso de esta tecnología.

Por último, reservé parte del cierre del *sprint* a tareas de verificación: investigué posibles errores tanto en el mapa como en la colección de imágenes,

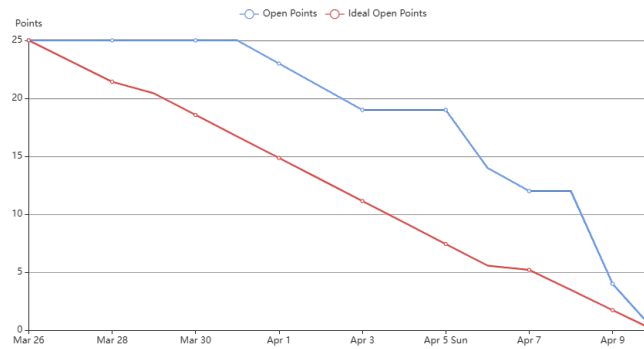


Figura A.7: Burndown Report del Sprint 14.

revisando el comportamiento en casos límite y buscando incoherencias derivadas de los cambios recientes.

Como se aprecia en la Figura A.7, el *burndown* refleja un inicio prácticamente plano, coherente con la menor disponibilidad de los primeros días. También se observa un ajuste en la línea ideal, debido a la ampliación del *sprint* en un día por motivos académicos. A partir de la mitad del periodo el descenso se acelera, mostrando cómo el cierre de tareas se concentró en los últimos días, hasta completar el *sprint* al final de la semana.

Sprint 15 - Integración de usuarios y administración

El *Sprint 15* se extendió durante casi dos semanas. Aunque inicialmente estaba planificado como un *sprint* de una semana, por diversos motivos se decidió aplazar el cierre una semana adicional en el último momento, lo que afectó directamente a la planificación y explica que el *burndown report* presente un comportamiento menos lineal de lo habitual.

Para este *sprint* estimé una dedicación total de 23 horas y finalmente invertí 25 horas y 30 minutos. Durante los primeros días apenas pude avanzar y, además, una parte importante del trabajo consistió en tareas estructurales que requerían acumular varios cambios antes de poder dar cierres completos, por lo que el progreso quedó más concentrado al final del periodo.

Los objetivos definidos para este *sprint* fueron los siguientes:

- Ajustar la interfaz del visor:
 - Cambiar el ancho del mapa para que ocupase toda la pantalla.

- Añadir un *checkbox* de “trazas dibujadas” en cada tesela.
- Modificar la *preview* de la zona seleccionada para que se almacenase directamente en memoria.
- Optimizar la lógica del *worker* para evitar comprobaciones continuas innecesarias.
- Diseñar el *mockup* completo de la funcionalidad de usuarios.
- Reestructurar la interacción con la base de datos, pasando de inserciones manuales a SQLAlchemy.
- Implementar el sistema de usuarios:
 - Crear pantallas de *login* y registro.
 - Crear un menú de administrador.
 - Definir permisos de acceso a pantallas en función del tipo de usuario.
 - Crear un panel de gestión de usuarios.
 - Crear un panel de gestión del modelo.

En los primeros días abordé cambios principalmente visuales y de comportamiento del visor. En primer lugar, ajusté el mapa para que ocupase todo el ancho disponible, mejorando la experiencia de navegación y evitando el desaprovechamiento de espacio en pantallas grandes. A continuación, añadí un indicador por tesela mediante un *checkbox* asociado a si las trazas estaban dibujadas, lo que facilitaba distinguir rápidamente el estado de cada elemento dentro de la vista. En paralelo, modifiqué la lógica de *preview* de la zona seleccionada para que se almacenase directamente en memoria, reduciendo dependencias con almacenamiento temporal y simplificando el flujo de consulta y representación asociados a esta funcionalidad.

Seguidamente, realicé ajustes internos para mejorar robustez. En concreto, modifiqué la lógica del *worker* para que no estuviese comprobando continuamente el estado de forma activa, sino que operase de manera más eficiente.

La segunda mitad del *sprint* se centró en la nueva funcionalidad de usuarios. Primero elaboré el *mockup* completo para tener claro el flujo de pantallas y el rol de cada perfil. A partir de ahí, reestructuré la interacción con base de datos: sustituí inserciones manuales por un enfoque basado en SQLAlchemy, de forma que el acceso a datos quedase más mantenible

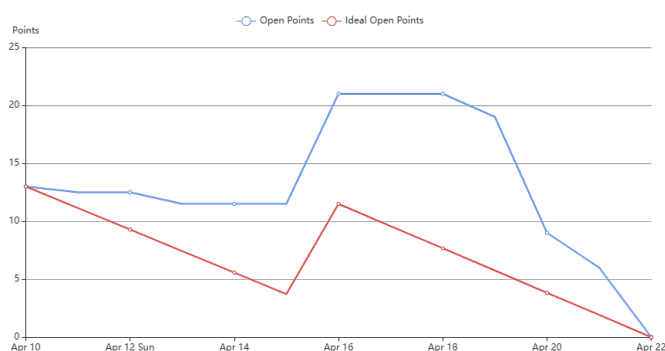


Figura A.8: Burndown Report del Sprint 15.

y coherente con el crecimiento del proyecto. Una vez estabilizada esta base, implementé las pantallas de *login* y registro y construí el menú de administrador.

Finalmente, desarrollé la capa de permisos, restringiendo el acceso a determinadas pantallas en función del tipo de usuario y del rol asignado, y creé los paneles de administración: uno para la gestión de usuarios y otro orientado a la gestión del modelo. Este bloque concentró buena parte del trabajo del *sprint*, ya que dependía de tener cerradas previamente la navegación, la persistencia y la estructura de roles.

Como se aprecia en la Figura A.8, el *burndown* presenta un comportamiento atípico. Por un lado, el descenso inicial es muy limitado, coherente con la baja disponibilidad de los primeros días y con el hecho de que varias tareas eran estructurales y no se podían cerrar de forma aislada. Por otro, se observa un reajuste a mitad del periodo: en ese punto se amplió el *sprint* una semana adicional y se replanificó la línea ideal, lo que explica el cambio de pendiente en el tramo central. A partir de ahí, el cierre de tareas se concentra al final, cuando ya quedaron consolidadas las dependencias internas y el *sprint* termina con una caída pronunciada hasta completarse el día 22 de abril.

Sprint 16 - Cierre de la aplicación

El *Sprint 16* se desarrolló del 23 al 30 de abril, con una duración de una semana. Para este *sprint* estimé una dedicación total de 22,5 horas, aunque finalmente invertí aproximadamente 25 horas. El foco principal de esta iteración fue consolidar la funcionalidad de usuarios y administración introducida en el *sprint* anterior, mejorar la usabilidad de la interfaz y

reforzar la gestión de modelos desde el panel de administrador, poniendo así fin a la funcionalidad principal de la aplicación.

Los objetivos definidos para este *sprint* fueron los siguientes:

- Arreglar la página de gestión de usuarios.
- Arreglar el desplegable del menú y su comportamiento.
- Cambiar el nombre de los *folds* y ajustar la lógica de búsqueda.
- Mejorar la gestión de *folds* desde administración:
 - Permitir que el administrador renombre *folds*.
 - Cargar nuevos *folds* y eliminar los existentes.
 - Validar nuevos modelos por *fold*.
- Añadir *tooltips* en las funciones del *home*.
- Añadir *tooltips* en la colección y la galería.
- Añadir lógica de pintado de trazas en el visor.
- Formatear la página de *log in*.
- Formatear la página de registro.
- Permitir la descarga de informes de usuarios en formato CSV y PDF.
- Realizar las traducciones de todas las pantallas.

En primer lugar, trabajé en corregir y estabilizar la parte de administración. Ajusté la página de gestión de usuarios, revisando tanto el comportamiento como la presentación, y arreglé el desplegable del menú, ya que en algunos flujos no se mostraban de forma consistente. A partir de ahí, me centré en la gestión del modelo, ya que era una de las piezas críticas del panel de administración: cambié el criterio de nombres y la lógica de búsqueda para que fuese más clara, permití que el administrador renombrase *folds*, añadí la funcionalidad de carga y eliminación de nuevos *folds*, todo ello con un nuevo proceso de validación por detrás.

En paralelo, realicé mejoras de usabilidad en la interfaz. Añadí *tooltips* en todas las pantalla que faltaban de emplear estos servicios, con el objetivo de guiar al usuario y reducir ambigüedades en acciones frecuentes. Además,

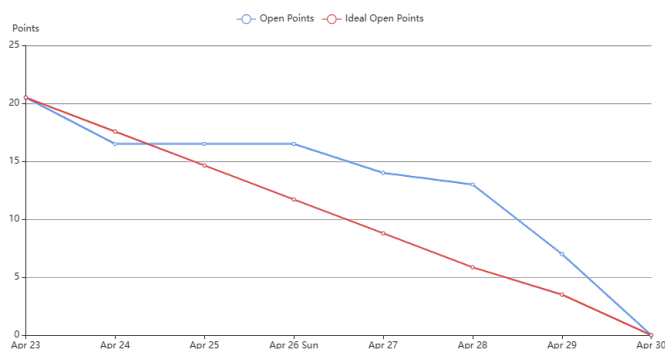


Figura A.9: Burndown Report del Sprint 16.

implementé la lógica de pintado de trazas directamente en el visor, para así finalizar el flujo del sistema.

También dediqué parte del *sprint* a mejorar la apariencia de las pantallas de autenticación, formateando tanto la página de *log in* como la de registro para que siguiesen el mismo estilo visual del resto de la aplicación. Como cierre de funcionalidad en el área de administración, añadí la posibilidad de descargar información de usuarios en formato CSV y PDF, facilitando la exportación de datos desde el panel.

Por último, completé la traducción de todas las pantallas, integrando los textos en el flujo de internacionalización para que la aplicación quedase consistente en los cinco disponibles.

Cabe señalar que, en los últimos días del *sprint*, se incorporaron dos tareas adicionales: exportación CSV/PDF y traducción completa de pantallas. Como consecuencia, tuve que prescindir de una tarea inicialmente prevista: cargar usuarios automáticamente al lanzar Flask, que quedó fuera del alcance de esta iteración para priorizar los cambios añadidos.

También, recalcar que, tras finalizar este sprint, lancé la segunda release de la aplicación, la 0.8, esto debido a que ya había logrado un buen flujo del sistema.

Como muestra la Figura A.9, el *burndown* refleja un descenso progresivo y estable durante la semana, con una ligera pérdida de alineación respecto a la línea ideal en el tramo final. Esto se debe a la incorporación de tareas en los últimos días, lo que incrementó la carga restante cuando el *sprint* ya estaba avanzado. Aun así, el cierre se completó el 30 de abril, consolidando el bloque final de la aplicación.

Sprint 17 - Arreglos funcionales y avances en la memoria

El *Sprint 17* se desarrolló del 1 al 7 de mayo, con una duración de una semana. Para este *sprint* estimé una dedicación inicial de 20 horas, aunque finalmente invertí aproximadamente 24 horas. Fue una iteración especialmente problemática, ya que gran parte del tiempo se destinó a modificar y depurar la BBDD, lo que bloqueó varias tareas que dependían directamente de ella y provocó que no pudiera cerrar tanto trabajo como estaba previsto.

Los objetivos definidos para este *sprint* fueron los siguientes:

- Realizar primeras comprobaciones con `pytest-cov`.
- Cambiar nombres confusos de la interfaz.
- Establecer los requisitos funcionales y no funcionales.
- Cambiar y actualizar la parte de Google Earth Engine (GEE) en la memoria.
- Incorporar funcionalidad de fotos para los usuarios.
- Redactar el apartado de técnicas y herramientas.
- Corregir un *bug* relacionado con las imágenes en la gestión de usuarios.
- Cargar usuarios cuando se lance Flask y revisar toda la BBDD.
- Permitir eliminar un usuario y sus parcelas en cascada.
- Hacer el servicio de *folds* en segundo plano y admitir cualquier nombre para los modelos.
- Redactar el manual del programador.

Durante los primeros días realicé algunos avances funcionales y de documentación. Por un lado, cambié nombres confusos de la interfaz para que las acciones fuesen más claras para el usuario y empecé a revisar la integración de `pytest-cov`, aunque esta parte no quedó consolidada hasta el *sprint* siguiente. Por otro lado, establecí los requisitos funcionales y no funcionales del sistema, lo que permitió dejar más claro el alcance real de la aplicación y su comportamiento esperado.

También avancé en la memoria, especialmente en dos frentes. Primero, actualicé la parte relativa a Google Earth Engine, ajustando la redacción para que fuese coherente con los cambios realizados en la funcionalidad geoespacial, ya que ya no usábamos este motor, y con las limitaciones detectadas en *sprints* anteriores. Después, redacté el apartado de técnicas y herramientas, donde se recogían las tecnologías principales utilizadas a lo largo del proyecto y su justificación dentro del desarrollo.

En paralelo, añadí la funcionalidad de fotos para los usuarios y corregí un *bug* relacionado con las imágenes en la página de gestión de usuarios. Aunque este error no era especialmente complejo, sí afectaba a la presentación y coherencia de la interfaz, por lo que era necesario resolverlo antes de seguir ampliando el módulo de administración.

Sin embargo, el mayor problema del *sprint* estuvo en la BBDD. Casi toda la iteración quedó condicionada por cambios y errores derivados de su estructura y de la forma en que interactuaba con las nuevas funcionalidades. Esto me impidió cerrar varias tareas previstas, como cargar usuarios al lanzar Flask, revisar completamente la BBDD, permitir eliminar usuarios junto con sus parcelas en cascada, y hacer que el servicio de *folds* funcionase en segundo plano admitiendo nombres arbitrarios para los modelos. Estas tareas quedaron aplazadas al *sprint* siguiente, donde se resolvería finalmente el bloqueo principal.

Además, quedó pendiente la redacción del manual del programador, ya que requería una revisión más estable del código y de la estructura final de la aplicación. En retrospectiva, durante este *sprint* fui resolviendo distintos flujos de error asociados a la BBDD, pero no los desglosé en tarjetas independientes, por lo que parte del esfuerzo realizado no quedó reflejado con tanta precisión en el tablero.

Como se observa en la Figura A.10, el *burndown* muestra un avance irregular. La línea de *open points* desciende poco durante buena parte del *sprint*, ya que muchas tareas dependían de resolver primero el problema de la BBDD. La bajada pronunciada del último día no representa tanto un cierre real de todas las tareas, sino principalmente el aplazamiento de varias de ellas al *sprint* siguiente, donde se abordarían una vez desbloqueada la base de datos.

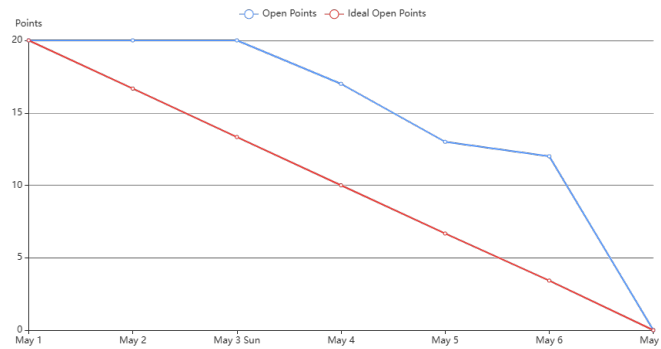


Figura A.10: Burndown Report del Sprint 17.

Sprint 18 - Definición de arquitecturas y arreglo final de la BBDD

El *Sprint 18* se desarrolló del 8 al 14 de mayo. Para este *sprint* estimé una dedicación total de 28 horas, y finalmente invertí aproximadamente 32 horas. El foco principal fue dejar estabilizada la base de datos, cerrando problemas que venían bloqueando tareas del *sprint* anterior, y definir de manera clara la arquitectura del sistema, tanto a nivel conceptual como mediante diagramas, además de reforzar la calidad del proyecto con *coverage* y SonarCloud.

Es importante aclarar que las tareas *Cargar usuarios cuando se lance Flask* y *revisar toda la BBDD*, *Permitir eliminar un usuario y sus parcelas en cascada*, *Hacer el servicio de folds en segundo plano y admitir el nombre que sea para los modelos* y *Primeras comprobaciones con pytest-cov* venían arrastradas del *sprint* anterior. No pude finalizarlas entonces porque, como bien he comentado, existía un problema en la BBDD que impedía avanzar con garantías, y dicho bloqueo quedó resuelto durante este *sprint*, lo que me permitió cerrarlas correctamente.

Los objetivos definidos para este *sprint* fueron los siguientes:

- Cargar usuarios cuando se lance Flask y revisar la BBDD (tarea arrastrada).
- Implementar el servicio de *folds* en segundo plano y admitir nombres arbitrarios para modelos (tarea arrastrada).
- Permitir eliminar un usuario y sus parcelas en cascada (tarea arrastrada).

- Primeras comprobaciones con *pytest-cov* (tarea arrastrada).
- Cubrir los flecos restantes del *coverage*.
- Configurar SonarCloud correctamente.
- Redactar el apartado F de anexos (ODS).
- Definir y documentar la arquitectura del sistema:
 - Crear una arquitectura hexagonal.
 - Crear el diagrama de arquitectura por capas.
 - Crear diagramas arquitectónicos adicionales que quedaban pendientes.
 - Redactar una explicación para este apartado (se pasó al siguiente *sprint*).
- Arreglar el diagrama de casos de uso y los diagramas de BBDD.
- Redactar aspectos relevantes que todavía faltaban en la memoria.
- Corregir un *bug* puntual de la imagen de perfil en la gestión de usuarios.

Durante los primeros días el trabajo se centró en desbloquear y consolidar la base de datos, ya que era el requisito previo para poder cerrar varias tareas que estaban pendientes. En esta fase revisé el estado general de la BBDD y dejé implementada la carga de usuarios al iniciar Flask, asegurando que el sistema arrancase con un estado coherente. A partir de ahí, pude retomar la parte de los modelos: implementé el servicio en segundo plano y adapté la lógica para que el sistema aceptase nombres de modelos sin restricciones artificiales. En paralelo, añadí también la eliminación en cascada de un usuario junto con sus parcelas.

A mitad del *sprint* surgió un *bug* menor relacionado con la imagen de perfil en la gestión de usuarios. Aunque no supuso un bloqueo grave, sí era necesario corregirlo para dejar la funcionalidad cerrada y estable, por lo que lo abordé en cuanto apareció.

Una vez estabilizada la base del sistema, el grueso del *sprint* se orientó a la definición de arquitecturas y a la mejora de calidad. Por un lado, trabajé en la parte de documentación técnica, creando la propuesta de arquitectura hexagonal y el diagrama por capas, además de generar otros diagramas arquitectónicos que aportasen una visión adicional al sistema. También

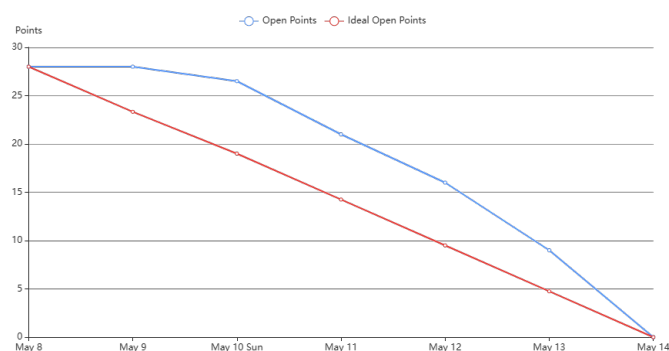


Figura A.11: Burndown Report del Sprint 18.

revisé y corregí el diagrama de casos de uso y los diagramas asociados a la BBDD para que fuesen consistentes con el estado real del sistema tras los cambios de este tramo.

Por otro lado, reforcé el apartado de calidad del proyecto: configuré SonarCloud correctamente y cerré los flecos restantes de *coverage*, asegurando que el proyecto quedase mejor respaldado por pruebas y con un análisis estático más limpio. Además, redacté el apartado F de anexos relativo a los ODS e incorporé algunos aspectos relevantes que todavía faltaban en la memoria.

Finalmente, aunque la tarea de *Redactar el apartado de arquitecturas* estaba inicialmente planificada para este *sprint*, se complicó más de lo esperado, ya que me llevó varias iteraciones decidir la mejor forma de presentar y justificar la arquitectura y su encaje con el proyecto. Por ello, dejé avanzados los diagramas y el criterio de diseño durante este *sprint*, pero la redacción final del apartado quedó pospuesta y se completaría el primer día del *sprint* siguiente.

Como se aprecia en la Figura A.11, el *burndown* muestra un inicio prácticamente plano, ya que varias tareas dependían de resolver primero el bloqueo asociado a la BBDD y de consolidar cambios estructurales antes de poder cerrar *issues*. Una vez resueltas esas dependencias, el cierre de tareas se aceleró de forma clara en la segunda mitad del *sprint*, completándose el conjunto de trabajo el 14 de mayo.

A.3. Estudio de viabilidad

Viabilidad económica

En este apartado se realiza una estimación económica del proyecto, considerando los principales recursos necesarios para su desarrollo, puesta en marcha y mantenimiento básico. Aunque el trabajo se ha realizado en un contexto académico y no persigue una explotación comercial inmediata, resulta conveniente valorar su coste aproximado como si fuese desarrollado dentro de un entorno profesional. Para ello, se tienen en cuenta los costes de personal, hardware, software, despliegue, mantenimiento e indirectos, así como los beneficios no económicos derivados de la aplicación.

La duración total del proyecto ha sido de aproximadamente siete meses y medio. Para el escenario base se toma como referencia un perfil de desarrollador *full-stack* junior avanzado, con conocimientos de Python, Flask, pruebas automáticas e integración de modelos de inferencia. Se considera un salario bruto anual de 30.000 €, situado entre las referencias salariales de un perfil junior Python y un perfil Python/Django en España, y ajustado al tipo de responsabilidades asumidas durante el desarrollo.

Costes de personal

El coste de personal constituye la partida principal del proyecto, ya que la mayor parte del esfuerzo se ha concentrado en el análisis del problema, el estudio del algoritmo base, el desarrollo de la aplicación web, la integración de la inferencia, la implementación del visor cartográfico, la gestión de usuarios y modelos, las pruebas y la documentación.

Para el cálculo del coste por hora se emplean las siguientes expresiones:

$$\text{Coste bruto por hora} = \frac{\text{Salario bruto anual}}{1,760} \quad (\text{A.1})$$

$$\text{Coste empresa por hora} = \text{Coste bruto por hora} \times (1 + 0,3057) \quad (\text{A.2})$$

$$\text{Coste de desarrollo} = \text{Horas dedicadas} \times \text{Coste empresa por hora} \quad (\text{A.3})$$

El factor 0,3057 representa una aproximación a las cotizaciones empresariales a la Seguridad Social, considerando contingencias comunes, desempleo, FOGASA, formación profesional y MEI.

Concepto	Porcentaje
Contingencias comunes	23,60 %
Desempleo	5,50 %
FOGASA	0,20 %
Formación profesional	0,60 %
MEI	0,67 %
Total aproximado	30,57 %

Tabla A.1: Cotizaciones empresariales consideradas para el cálculo del coste de personal.

En el caso del desarrollador principal, se estima una dedicación aproximada de 900 horas. Para la labor de supervisión se considera una dedicación de 30 horas por cada tutor, asociada a reuniones, revisión de avances, orientación técnica y seguimiento del proyecto.

Persona	Rol	Horas	Coste empresa/h	Coste
Marcos Zamorano Lasso	Desarrollador	900	22,26 €	20.030,63 €
Álvar Arnaiz González	Supervisor	30	33,38 €	1.001,53 €
David Martínez Acha	Supervisor	30	33,38 €	1.001,53 €
Total		960		22.033,69 €

Tabla A.2: Estimación de costes de personal del proyecto.

Costes de hardware

Para el desarrollo del proyecto se ha utilizado un equipo portátil personal, suficiente para implementar la aplicación, ejecutar el entorno Flask, realizar pruebas locales y llevar a cabo inferencias de validación. Aunque no ha sido necesario adquirir hardware específico, se imputa una parte proporcional del coste del equipo en función de su vida útil estimada.

El coste de hardware se calcula mediante la expresión:

$$\text{Coste hardware} = \frac{\text{Precio del equipo}}{\text{Vida útil}} \times \frac{\text{Meses del proyecto}}{12} \quad (\text{A.4})$$

Considerando un equipo valorado en 1.200 €, una vida útil de 5 años y una duración del proyecto de 7,5 meses, se obtiene:

$$\text{Coste hardware} = \frac{1,200}{5} \times \frac{7,5}{12} = 150,00 \text{ €} \quad (\text{A.5})$$

Elemento	Precio	Vida útil	Coste imputado
Equipo portátil de desarrollo	1.200 €	5 años	150,00 €
Total			150,00 €

Tabla A.3: Costes de hardware imputados al proyecto.

Costes de software

En cuanto al software, se ha priorizado el uso de herramientas gratuitas, de código abierto o disponibles sin coste adicional para el desarrollo académico. Esto incluye el lenguaje Python, el *framework* Flask, SQLite, SQLAlchemy, PyTorch, Tailwind CSS, DaisyUI, Leaflet, Git, GitHub, Visual Studio Code, *diagrams.net*, TeXstudio, MiKTeX y Jupyter Notebook, entre otras herramientas utilizadas durante el desarrollo y la documentación.

Software	Uso principal	Coste
Python, Flask y extensiones	Desarrollo de la aplicación web y backend	0,00 €
PyTorch y librerías de inferencia	Ejecución de modelos de segmentación	0,00 €
SQLite y SQLAlchemy	Persistencia y acceso a datos	0,00 €
Tailwind CSS, DaisyUI y Leaflet	Interfaz web y visor cartográfico	0,00 €
Git, GitHub y Zube	Control de versiones y gestión del proyecto	0,00 €
TeXstudio, MiKTeX y diagrams.net	Documentación y diagramas	0,00 €
Visual Studio Code y Jupyter Notebook	Desarrollo, pruebas y análisis del algoritmo	0,00 €
Total		0,00 €

Tabla A.4: Costes de software considerados en el proyecto.

Por tanto, el coste directo de software se considera nulo. No obstante, esta estimación no implica que el software carezca de valor, sino que las herramientas utilizadas no han requerido pagos adicionales durante el desarrollo del TFG.

Costes de despliegue y mantenimiento

Aunque la aplicación puede ejecutarse en local durante el desarrollo y se despliega en una infraestructura universitaria, se estima un escenario de referencia basado en un despliegue básico en un servidor privado virtual. Este escenario permite aproximar el coste que tendría mantener la aplicación disponible durante el periodo equivalente al desarrollo.

Además del alojamiento, debe considerarse una dedicación mensual de mantenimiento para tareas como actualización de dependencias, revisión de logs, copias de seguridad, limpieza de ficheros generados y supervisión del servicio mediante Gunicorn. Se estima una dedicación media de 1,5 horas mensuales durante 7,5 meses.

Concepto	Coste unitario	Periodo	Coste
Servidor VPS básico	12,00 €/mes	7,5 meses	90,00 €
Dominio web	12,00 €/año	7,5 meses	7,50 €
Almacenamiento y copias de seguridad	5,00 €/mes	7,5 meses	37,50 €
Mantenimiento técnico	22,26 €/h	11,25 h	250,38 €
Total			385,38 €

Tabla A.5: Estimación de costes de despliegue y mantenimiento.

Costes indirectos

Además de los costes directos, se consideran costes indirectos asociados al desarrollo del proyecto. En esta categoría se incluyen gastos como electricidad, conexión a internet, infraestructura doméstica o universitaria, material auxiliar y otros recursos de apoyo no imputados de forma individual.

Siguiendo una estimación habitual en este tipo de estudios, se aplica un 15 % sobre el total de costes directos:

$$\text{Costes indirectos} = \text{Costes directos} \times 0,15 \quad (\text{A.6})$$

Tipo de coste	Importe
Costes de personal	22.033,69 €
Costes de hardware	150,00 €
Costes de software	0,00 €
Costes de despliegue y mantenimiento	385,38 €
Costes directos	22.569,07 €
Costes indirectos, 15 %	3.385,36 €
Coste total estimado	25.954,43 €

Tabla A.6: Resumen de costes económicos del proyecto.

Beneficios

No se esperan ingresos comerciales directos derivados del proyecto, ya que su finalidad es puramente académica e investigadora. La aplicación se ha concebido como un prototipo funcional orientado a demostrar la viabilidad de integrar modelos de segmentación semántica en un entorno web accesible, no como un producto comercial cerrado.

No obstante, sí existen beneficios intangibles relevantes. En primer lugar, el sistema puede servir como apoyo a investigaciones relacionadas con el análisis de trazas de herbivoría y presión ecológica. Adicionalmente, automatiza parcialmente una tarea que, de forma manual, resultaría costosa, lenta y difícil de escalar.

Conclusión

Desde el punto de vista económico, el proyecto resulta viable en un contexto académico, principalmente porque se ha apoyado en herramientas gratuitas, bibliotecas de código abierto y recursos hardware ya disponibles. La partida más relevante corresponde al coste de personal, algo esperable en un proyecto software donde el mayor valor se encuentra en el análisis, diseño, implementación, pruebas y documentación.

El coste total estimado asciende a 25.954,43 €, considerando personal, hardware, software, despliegue, mantenimiento e indirectos. Aunque no se prevén beneficios comerciales directos, el proyecto aporta valor investigador, académico y técnico, además de constituir una base sólida para futuras

líneas de desarrollo relacionadas con la monitorización ecológica mediante imágenes aéreas y modelos de visión por computador.

Viabilidad legal

Apéndice *B*

Especificación de Requisitos

B.1. Introducción

Este apéndice recoge la Especificación de Requisitos del sistema desarrollado, con el objetivo de dejar formalizado qué funcionalidades debe proporcionar la aplicación y bajo qué condiciones de uso deben ejecutarse. Para ello, se parte de los objetivos del proyecto y del flujo de interacción previsto, estructurando la información en un catálogo de requisitos funcionales y no funcionales, así como en una especificación detallada basada en casos de uso.

La especificación se centra en el comportamiento observable desde la interfaz web y en las restricciones asociadas a los distintos perfiles de usuario. En particular, se identifican los actores del sistema, se enumeran los requisitos que guían el desarrollo y, finalmente, se describen los casos de uso que articulan el *pipeline* de la aplicación, desde la selección de una imagen y el cálculo de trazas hasta la visualización, persistencia y explotación del resultado.

B.2. Objetivos generales

Los objetivos generales del proyecto, desde el punto de vista del producto software a construir, pueden sintetizarse en los siguientes puntos:

1. Desarrollar una aplicación web que permita cargar imágenes y ejecutar un proceso de detección y segmentación de trazas, ofreciendo una visualización clara del resultado superpuesto sobre la imagen original.

2. Integrar en el *backend* la lógica de inferencia proporcionada por el modelo de segmentación, de modo que el procesamiento quede debidamente configurado para su consumo desde el cliente.
3. Definir un sistema de usuarios y roles que module el acceso a funcionalidades, distinguiendo entre uso básico sin autenticación, uso avanzado con persistencia y operaciones restringidas, y un rol administrador orientado a la configuración interna del sistema.
4. Facilitar la explotación del resultado mediante acciones complementarias, como la descarga de resultados, permitiendo repetir análisis sobre nuevas entradas de manera eficiente.
5. Lograr la implementación de un módulo GIS que habilite el trabajo por zona geográfica, permitiendo seleccionar una zona, generar un *grid* de imágenes asociadas a esa zona y crear una galería con las diferentes zonas e imágenes generadas.

Dado que el núcleo del proyecto es la aplicación web y su *pipeline* de interacción, este apéndice se centra en concretar, con el mayor grado de precisión posible, los requisitos que debe satisfacer el sistema y las limitaciones bajo las que operan sus funcionalidades.

B.3. Catálogo de requisitos

Este catálogo establece los requisitos del sistema en dos categorías complementarias. Por un lado están los requisitos funcionales, que describen qué servicios y operaciones debe ofrecer la aplicación a cada tipo de actor. Por otro lado, los requisitos no funcionales, que recogen restricciones de calidad, seguridad y operatividad condicionando así cómo se implementan dichas funcionalidades.

Requisitos funcionales

- **RF-01 Procesamiento de imagen:** la aplicación debe permitir ejecutar el flujo completo de análisis sobre una ortoimagen, desde su selección hasta la obtención y representación del resultado.
 - **RF-01.1 Insertar imagen:** el usuario debe poder seleccionar una imagen y previsualizarla en el cliente.

- **RF-01.2 Ejecutar detección y segmentación:** el sistema debe enviar la imagen al servidor cuando proceda, ejecutar la inferencia con el modelo configurado y generar la salida asociada.
 - **RF-01.3 Visualizar resultado:** el cliente debe poder representar las trazas superpuestas sobre la imagen original, permitiendo su visualización.
 - **RF-01.4 Borrar imagen:** el usuario debe poder reiniciar el flujo eliminando la imagen activa y los resultados asociados, devolviendo la aplicación a un estado inicial consistente.
- **RF-02 Exportación de resultados:** el sistema debe permitir la descarga del resultado generado en un formato reutilizable, asociado a la imagen analizada.
 - **RF-03 Gestión de cuentas:** la aplicación debe incorporar un sistema de usuarios que habilite funcionalidades adicionales bajo sesión autenticada.
 - **RF-03.1 Registro:** el usuario anónimo debe poder crear una cuenta proporcionando los datos requeridos.
 - **RF-03.2 Inicio de sesión:** el usuario debe poder autenticarse mediante credenciales válidas para acceder a funcionalidades restringidas.
 - **RF-03.3 Cierre de sesión:** el usuario autenticado debe poder finalizar su sesión de forma explícita.
 - **RF-03.4 Administración de perfil:** el usuario autenticado debe poder consultar y actualizar la información básica de su perfil dentro de los límites establecidos por el sistema.
 - **RF-04 Integración GIS:** la aplicación debe ofrecer un módulo cartográfico para trabajar sobre una zona geográfica seleccionada por el usuario autenticado.
 - **RF-04.1 Acceso al módulo GIS:** el usuario autenticado debe poder acceder al visor y a las funcionalidades cartográficas disponibles.
 - **RF-04.2 Recorte de zona de interés:** el sistema debe permitir definir una zona de trabajo mediante una geometría válida, utilizada como delimitación operativa.

- **RF-04.3 Visualización de trazas en la zona:** el sistema debe permitir representar las trazas asociadas a la zona seleccionada, una vez calculadas, manteniendo consistencia con la georreferenciación del visor.
- **RF-05 Administración:** la aplicación debe incorporar un rol administrador con capacidades de gestión y configuración interna.
 - **RF-05.1 Administrar modelos:** el administrador debe poder gestionar la configuración del modelo de detección y segmentación, incluyendo la selección del modelo activo y la opción de añadir o eliminar más modelos operativos, tras ser validados por el sistema.
 - **RF-05.2 Gestionar usuarios:** el administrador debe poder consultar el listado de usuarios y ejecutar operaciones de gestión sobre sus cuentas conforme a las políticas del sistema.

Requisitos no funcionales

- **RNF-01 Usabilidad y consistencia de interfaz:** la aplicación debe ofrecer un flujo de uso claro, con retroalimentación explícita ante operaciones de cálculo, descarga y error, manteniendo coherencia visual entre pantallas.
- **RNF-02 Accesibilidad y compatibilidad:** la interfaz debe ser accesible desde navegador web y funcionar de forma consistente en navegadores modernos, evitando dependencias de tecnologías no soportadas.
- **RNF-03 Seguridad de autenticación y autorización:** las funcionalidades restringidas deben requerir sesión autenticada, y el acceso a operaciones de administración debe estar limitado al rol administrador, evitando exposición de información sensible en pantallas o exportaciones.
- **RNF-04 Aislamiento y privacidad de datos:** los recursos persistentes, como zonas, imágenes y resultados, deben quedar asociados al usuario propietario, de modo que no sean accesibles por terceros sin permisos.
- **RNF-05 Robustez y tratamiento de errores:** el sistema debe validar entradas, rechazar archivos inválidos, controlar tamaños máximos de subida y manejar fallos durante la inferencia o la consulta

de fuentes externas, informando al usuario y manteniendo el estado consistente.

- **RNF-06 Integridad y trazabilidad de persistencia:** la base de datos y el sistema de ficheros deben permanecer coherentes, garantizando correspondencia entre entradas y resultados, de modo que la aplicación pueda recuperar y representar correctamente el estado de cada operación.
- **RNF-07 Rendimiento:** la visualización de trazas debe ser eficiente y evitar cargas innecesarias, y la inferencia debe ejecutarse de forma que el usuario reciba feedback de progreso y se minimicen bloqueos perceptibles.
- **RNF-08 Mantenibilidad, extensibilidad y testabilidad:** el sistema debe permitir evoluciones incrementales sin reescrituras globales, manteniendo contratos estables entre cliente y servidor, y disponiendo de pruebas que reduzcan el riesgo de regresión en funcionalidades críticas.
- **RNF-09 Internacionalización:** los textos visibles deben poder presentarse en más de un idioma mediante una estrategia centralizada de traducciones, manteniendo consistencia en plantillas y recursos del cliente.
- **RNF-10 Cumplimiento legal y licencias de datos:** la aplicación debe basar la adquisición y uso de datos geoespaciales en fuentes y términos de reutilización compatibles con el análisis automatizado, evitando dependencias de contenidos con restricciones incompatibles con el objetivo del proyecto.

B.4. Especificación de requisitos

En esta sección se presenta la especificación funcional de la aplicación mediante casos de uso, describiendo de forma estructurada las interacciones que el usuario puede realizar a través de la interfaz. Dado que la aplicación se consume desde el cliente web, el diagrama sintetiza las funcionalidades visibles para cada tipo de actor, mientras que el *backend* se considera un soporte imprescindible para ejecutar la inferencia, gestionar el estado de las imágenes y servir los resultados, sin modelarse explícitamente como actor independiente.

Diagrama de Casos de Uso

En la figura B.1 se recoge el diagrama general de casos de uso y, a continuación, se detallan las descripciones individuales de cada caso representado. Estas tablas se han derivado a partir del flujo de uso previsto y de los *moc-kups* de la interfaz, con el objetivo de concretar precondiciones, secuencias principales, relaciones entre funcionalidades y el nivel de importancia de cada operación dentro del *pipeline* de la aplicación.

B.5. Actores del sistema

En esta sección se describen los distintos actores que interactúan con la aplicación web de detección de trazas. Los actores representan los perfiles de usuario y roles funcionales que acceden al sistema, diferenciándose en función de sus permisos y capacidades dentro de la aplicación. Esta distinción permite modelar de forma clara las responsabilidades y el alcance de cada tipo de usuario.

Usuario Anónimo

El **Usuario Anónimo** representa a cualquier persona que accede a la aplicación sin haber iniciado sesión ni disponer de una cuenta registrada. Este actor tiene acceso a las funcionalidades básicas del sistema, orientadas a la demostración y uso general de la herramienta de detección.

Concretamente, el Usuario Anónimo puede:

- Cargar una imagen para su análisis.
- Ejecutar el proceso de detección y segmentación de trazas.
- Visualizar el resultado obtenido.
- Descargar los resultados obtenidos tras el proceso de detección y segmentación.
- Borrar la imagen cargada y reiniciar el flujo de procesamiento.
- Registrarse en el sistema.

Este perfil permite el uso inmediato de la aplicación sin barreras de acceso, manteniendo restringidas aquellas funcionalidades que requieren persistencia de datos o integración con servicios externos.

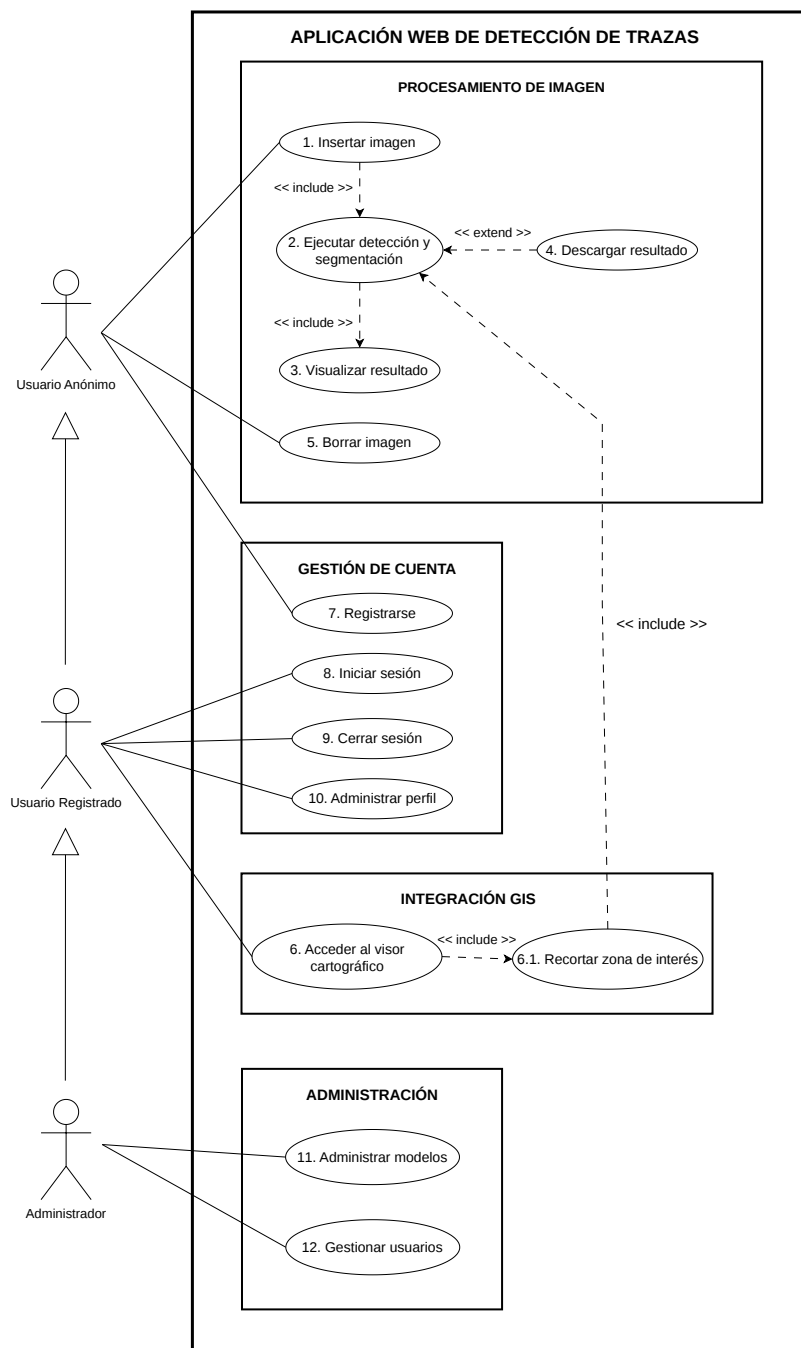


Figura B.1: Diagrama general de casos de uso de la aplicación

Usuario Registrado

El **Usuario Registrado** es un agente que dispone de una cuenta en el sistema y ha iniciado sesión correctamente. Este actor hereda todas las capacidades del Usuario Anónimo y, adicionalmente, puede acceder a funcionalidades avanzadas que requieren autenticación.

Además de las acciones disponibles para el Usuario Anónimo, el Usuario Registrado puede:

- Iniciar y cerrar sesión en el sistema.
- Administrar su perfil de usuario.
- Acceder al módulo de integración GIS mediante el visor.
- Definir una zona de interés y visualizar trazas dentro de dicha zona.

Este actor representa el perfil principal de uso de la aplicación, proporcionando acceso a funcionalidades de mayor valor añadido y a la integración con servicios externos.

Administrador

El **Administrador** es un usuario registrado con privilegios adicionales orientados a la gestión y mantenimiento del sistema. Este actor hereda todas las capacidades del Usuario Registrado y dispone de permisos específicos para la configuración interna de la aplicación.

Las funciones exclusivas del Administrador incluyen:

- Administrar y modificar los usuarios registrados en el sistema.
- Administrar los modelos de detección utilizados por el sistema.
- Seleccionar el modelo activo que se empleará en el proceso de detección y segmentación.

La existencia de este rol permite separar claramente las tareas de uso del sistema de aquellas relacionadas con su configuración y control, facilitando la mantenibilidad y escalabilidad de la aplicación.

Explicación de los casos de uso

En las Tablas B.1–B.13 se presenta la descripción detallada de los casos de uso identificados, incluyendo para cada uno los actores implicados, la precondition necesaria, el flujo principal de ejecución, las excepciones más relevantes, las relaciones de inclusión o extensión con otros casos y su nivel de importancia. Esta especificación permite trazar de forma directa las funcionalidades del sistema y clarificar el comportamiento esperado en situaciones normales y de error.

CU-01 Insertar imagen (previsualizar)	
Actores	Usuario anónimo, Usuario registrado, Administrador
Descripción	Permite al usuario seleccionar una imagen desde su dispositivo y previsualizarla en el cliente. La imagen no se envía al backend en este paso.
Precondición	El usuario ha accedido a la interfaz de procesamiento.
Flujo principal	1) El usuario pulsa <i>Insertar imagen</i> o la arrastra y selecciona un fichero válido. 2) El sistema carga la imagen en el cliente y la muestra en pantalla como previsualización. 3) El sistema actualiza el estado indicando que puede ejecutarse el cálculo de trazas.
Postcondición	Imagen seleccionada y visible en la interfaz.
Excepciones	E1) El usuario cancela la selección del fichero → no se modifica el estado de la interfaz. E2) El fichero no es un formato permitido → no se carga la previsualización y se informa al usuario. E3) Error al leer el fichero → se notifica el error y no se actualiza la imagen mostrada.
Relaciones	Incluye a CU-02 Ejecutar detección y segmentación.
Importancia	Alta

CU-02 Ejecutar detección y segmentación

Actores	Usuario anónimo, Usuario registrado, Administrador
Descripción	Ejecuta el cálculo de trazas mediante el modelo de detección y segmentación. La operación puede aplicarse sobre una única imagen cargada por el usuario o sobre una colección de imágenes generada a partir de una cuadrícula en el visor cartográfico. En el primer caso se procesa una imagen individual; en el segundo, el sistema procesa las teselas asociadas a la zona seleccionada.
Precondición	Existe una imagen seleccionada en el cliente (CU-01), una imagen previamente almacenada en el servidor en sesión, o una colección de imágenes/teselas generada desde una zona de interés definida en el visor cartográfico (CU-06.1).
Flujo principal	1) El usuario solicita el cálculo de trazas, bien desde el flujo de imagen individual o bien tras generar una cuadrícula de teselas en el visor cartográfico. 2) El sistema determina el tipo de entrada disponible: una imagen individual o una colección de imágenes asociada a una zona. 3) Si se trata de una imagen individual, el sistema envía o reutiliza la imagen correspondiente, valida el formato, la almacena si procede y ejecuta la inferencia sobre dicha imagen. 4) Si se trata de una colección, el sistema registra las imágenes o teselas asociadas, mantiene su estado de procesamiento y ejecuta la inferencia sobre cada una de ellas. 5) El backend aplica el modelo activo de detección y segmentación, generando el resultado de trazas correspondiente. 6) El sistema persiste el JSON de trazas asociado a la imagen individual o a cada imagen de la colección. 7) El sistema deja disponible la visualización del resultado (CU-03), ya sea sobre la imagen procesada o sobre las imágenes pertenecientes a la colección.
Postcondición	Trazas calculadas y disponibles para su visualización, bien para una imagen individual o bien para las imágenes de una colección generada desde el visor.
Excepciones	E1) No existe imagen seleccionada, imagen almacenada, ni colección de imágenes disponible → se informa al usuario y no se ejecuta el cálculo. E2) Formato no permitido o imagen no válida → se rechaza la operación y se informa al usuario. E3) Error durante la inferencia o la segmentación → se muestra un mensaje de error y no se generan trazas para la imagen afectada. E4) La carga excede el tamaño máximo permitido → se aborta la petición y se notifica el error. E5) En una colección, una o varias teselas no pueden procesarse correctamente → se marca su estado como fallido y se permite su posterior reintento cuando proceda.
Relaciones	Incluye CU-03 Visualizar resultado. Puede ser invocado tras CU-01 Insertar imagen o tras CU-06.1 Recortar zona de interés, cuando la zona seleccionada genera una colección de imágenes para su procesamiento.
Importancia	Alta

CU-03 Visualizar resultado

Actores	Usuario anónimo, Usuario registrado, Administrador
Descripción	Permite visualizar el resultado de la detección y segmentación superpuesto sobre la imagen original.
Precondición	El proceso de detección se ha ejecutado correctamente (CU-02).
Flujo principal	1) El sistema solicita el JSON de trazas al backend. 2) El sistema renderiza la imagen y superpone las trazas sobre un canvas. 3) El usuario visualiza el resultado por pantalla.
Postcondición	Resultado visualizado en pantalla.
Excepciones	E1) No existen trazas calculadas → se informa al usuario y no se dibujan trazas. E2) El archivo de trazas no se encuentra o está corrupto → se muestra error y se solicita recalcular. E3) Error de red o de timeout en la obtención del JSON → se notifica y se permite reintentar.
Relaciones	Incluido por CU-02 Ejecutar detección y segmentación.
Importancia	Alta

CU-04 Descargar resultado

Actores	Usuario anónimo, Usuario registrado, Administrador
Descripción	Permite descargar el resultado de la detección y segmentación para su uso externo.
Precondición	El resultado ha sido generado y visualizado (CU-03).
Flujo principal	1) El usuario solicita la descarga del resultado. 2) El sistema genera el archivo correspondiente. 3) El archivo es descargado por el usuario.
Postcondición	Resultado almacenado localmente por el usuario.
Excepciones	E1) No existe resultado disponible → se informa y se cancela la descarga. E2) Error generando el recurso descargable → se notifica y se permite reintentar.
Relaciones	Extiende CU-02 Ejecutar detección y segmentación.
Importancia	Media-Alta

CU-05 Borrar imagen

Actores	Usuario anónimo, Usuario registrado, Administrador
Descripción	Permite eliminar la imagen cargada y reiniciar el flujo de procesamiento.
Precondición	Existe una imagen en previsualización local o un resultado generado.
Flujo principal	1) El usuario solicita borrar la imagen. 2) Si solo existe previsualización local, el sistema limpia la imagen y el canvas en el cliente. 3) Si existe imagen en servidor, el sistema solicita al backend la eliminación de la imagen y sus trazas asociadas. 4) El sistema vuelve al estado inicial.
Postcondición	Sistema preparado para una nueva ejecución.
Excepciones	E1) No existe ninguna imagen para borrar → se informa al usuario. E2) Error eliminando recursos en servidor → se notifica y se mantiene el estado consistente. E3) Error de comunicación con el backend → se informa y se permite reintentar.
Importancia	Media

CU-06 Acceder al visor cartográfico

Actores	Usuario registrado, Administrador
Descripción	Permite acceder al módulo de integración con <i>leaflet</i> , <i>OSM</i> y <i>PNOA</i> para trabajar sobre una zona geográfica.
Precondición	El usuario ha iniciado sesión.
Flujo principal	1) El usuario accede al módulo GIS. 2) El sistema establece conexión con OSM y PNOA. 3) Se habilitan las funcionalidades de recorte y visualización.
Postcondición	Sesión GIS activa.
Excepciones	E1) Error de autenticación o de autorización con los servicios → se deniega el acceso y se informa al usuario. E2) Servicio no disponible o error de red → se notifica indisponibilidad temporal. E3) Configuración incompleta → se informa y se bloquea el acceso al módulo.
Relaciones	Incluye CU-06.1 Recortar zona de interés y CU-06.2 Visualizar trazas en la zona.
Importancia	Media-Alta

CU-06.1 Recortar zona de interés

Actores	Usuario registrado, Administrador
Descripción	Permite definir una zona geográfica de interés mediante un recorte sobre el visor.
Precondición	Acceso al visor (CU-06).
Flujo principal	1) El usuario define el área de recorte. 2) El sistema valida la geometría. 3) La zona queda establecida como área activa.
Excepciones	E1) Geometría inválida → se rechaza el recorte y se solicita corrección. E2) Zona fuera de límites permitidos → se informa y se requiere ajuste. E3) Error comunicándose con el servicio → se notifica y se permite reintentar.
Postcondición	Zona de interés definida.
Importancia	Media

CU-07 Registrarse

Actores	Usuario anónimo
Descripción	Permite crear una cuenta de usuario para acceder a funcionalidades restringidas del sistema.
Precondición	El usuario no tiene sesión iniciada.
Flujo principal	1) El usuario accede al formulario de registro. 2) El usuario introduce los datos requeridos. 3) El sistema valida y crea la cuenta.
Postcondición	Cuenta registrada y disponible para iniciar sesión.
Excepciones	E1) Datos inválidos o incompletos → se informa y no se crea la cuenta. E2) Usuario o correo ya existentes → se rechaza el registro. E3) Error interno de persistencia → se notifica y se permite reintentar.
Importancia	Media

CU-08 Iniciar sesión

Actores	Usuario registrado, Administrador
Descripción	Permite autenticar al usuario y habilitar las funcionalidades según su rol.
Precondición	El usuario dispone de credenciales válidas.
Flujo principal	1) El usuario introduce credenciales. 2) El sistema valida las credenciales. 3) El sistema inicia sesión y aplica permisos.
Postcondición	Sesión iniciada con permisos asociados al rol.
Excepciones	E1) Credenciales incorrectas → se informa y no se inicia sesión. E2) Cuenta deshabilitada o no verificada → se deniega el acceso. E3) Error interno de autenticación → se notifica y se permite reintentar.
Importancia	Media

CU-09 Cerrar sesión

Actores	Usuario registrado, Administrador
Descripción	Finaliza la sesión activa del usuario.
Precondición	Sesión iniciada previamente.
Flujo principal	1) El usuario solicita cerrar sesión. 2) El sistema invalida la sesión y revoca permisos. 3) El sistema redirige al estado de usuario anónimo.
Postcondición	Sesión finalizada.
Excepciones	E1) Sesión ya expirada → el sistema fuerza estado anónimo sin error. E2) Error interno al cerrar sesión → se notifica y se fuerza cierre.
Importancia	Baja-Media

CU-10 Administrar perfil

Actores	Usuario registrado, Administrador
Descripción	Permite consultar y actualizar la información básica del perfil del usuario.
Precondición	Sesión iniciada.
Flujo principal	1) El usuario accede a su perfil. 2) El sistema muestra la información disponible. 3) El usuario modifica los campos permitidos. 4) El sistema valida y guarda los cambios.
Postcondición	Perfil actualizado, si procede.
Excepciones	E1) Datos inválidos → se rechazan cambios y se informa. E2) Error de persistencia → se notifica y se mantiene el perfil anterior. E3) Sesión no válida → se solicita iniciar sesión.
Importancia	Baja-Media

CU-11 Administrar modelos

Actores	Administrador
Descripción	Permite al administrador gestionar los modelos de detección y segmentación disponibles en el sistema, incluyendo la consulta de folds existentes, la selección del modelo activo, la incorporación validada de nuevos modelos, el renombrado y la eliminación controlada de modelos no activos.
Precondición	Sesión iniciada como administrador.
Flujo principal	1) El administrador accede al módulo de gestión de modelos. 2) El sistema muestra el listado de modelos disponibles, indicando cuál se encuentra activo. 3) El administrador selecciona la operación que desea realizar: activar, renombrar, añadir o eliminar un modelo. 4) Si se añade un nuevo modelo, el administrador introduce un nombre válido y selecciona el archivo correspondiente. 5) El sistema valida la operación solicitada y, en caso de subida, comprueba que el archivo corresponde a un modelo compatible con el pipeline de inferencia. 6) Si la validación es correcta, el sistema persiste los cambios y actualiza el listado de modelos.
Postcondición	Listado y/o configuración de modelos actualizado, manteniendo un único modelo activo válido para el sistema.
Excepciones	E1) Selección inválida → se rechaza el cambio y se informa. E2) Modelo o pesos no disponibles → se notifica y se mantiene la configuración previa. E3) Error guardando la configuración → se informa y no se aplican cambios. E4) El archivo subido no corresponde a un modelo compatible → se rechaza la incorporación y no se modifica el listado de modelos. E5) El nombre asignado al modelo ya existe → se informa al administrador y se cancela la operación. E6) El administrador intenta eliminar el modelo activo → se rechaza la operación y se solicita activar previamente otro modelo. E7) Error durante la validación técnica del modelo → se elimina el archivo temporal y se mantiene la configuración previa.
Importancia	Media-Alta

CU-12 Gestionar usuarios

Actores	Administrador
Descripción	Permite al administrador realizar tareas de administración sobre las cuentas de usuario del sistema, incluyendo la consulta y búsqueda del listado, la visualización de detalles, la modificación de datos básicos y rol, la eliminación controlada de usuarios y la exportación del listado en formatos CSV y PDF.
Precondición	El actor ha iniciado sesión y dispone de permisos de administrador.
Flujo principal	1) El administrador accede a la sección de gestión de usuarios. 2) El sistema muestra el listado de usuarios registrados, junto con herramientas de búsqueda y exportación. 3) El administrador puede filtrar el listado, consultar el detalle de un usuario, editar sus datos, eliminarlo si está permitido o exportar la información disponible. 4) En operaciones de modificación o eliminación, el sistema valida los permisos y los datos introducidos. 5) El sistema persiste los cambios cuando proceda o genera el archivo de exportación solicitado. 6) El sistema confirma la operación y actualiza el listado cuando corresponde.
Postcondición	Información de usuario actualizada, usuario eliminado cuando proceda, o listado exportado en el formato solicitado, manteniendo la integridad de las cuentas protegidas.
Excepciones	E1) Usuario objetivo inexistente o no accesible → se informa al administrador y se cancela la operación. E2) Operación no permitida → se rechaza el cambio y se notifica. E3) Datos inválidos en la modificación → se rechazan cambios y se solicita corrección. E4) Error interno de persistencia o comunicación con la base de datos → se notifica el error y no se aplican cambios. E5) El administrador intenta eliminar una cuenta protegida o su propia cuenta → se rechaza la operación y se informa. E6) Error generando el archivo CSV o PDF → se notifica el fallo y no se descarga el recurso. E7) Parámetros de búsqueda inválidos → el sistema aplica valores por defecto o informa del problema.
Importancia	Media-Alta

Apéndice C

Especificación de diseño

C.1. Introducción

En este apéndice se presenta la especificación de diseño de la aplicación desarrollada, describiendo las decisiones estructurales que permiten transformar los requisitos definidos previamente en una solución software concreta. Para ello, se abordan de forma complementaria el diseño de datos, donde se modela la información persistente mediante los esquemas entidad-relación y relacional; el diseño procedimental, centrado en los principales flujos de interacción entre el usuario y el sistema; el diseño arquitectónico, orientado a explicar la organización general de la aplicación web monolítica modular basada en Flask y SQLite; y el diseño web, donde se recogen los *mockups* que guían la construcción de la interfaz. El objetivo es justificar cómo se ha organizado el sistema para garantizar coherencia, trazabilidad y mantenibilidad, conectando los requisitos funcionales y no funcionales con las decisiones técnicas adoptadas durante el desarrollo.

C.2. Diseño de datos

El diseño de datos define la estructura de persistencia que soporta el funcionamiento de la aplicación, garantizando que la información se almacene de forma coherente, trazable y escalable. En este apartado se presenta el esquema lógico de la base de datos, con el objetivo de justificar las entidades principales del dominio, sus relaciones y las restricciones de integridad que permiten mantener consistencia entre usuarios, parcelas, imágenes procesadas y resultados generados por el sistema.

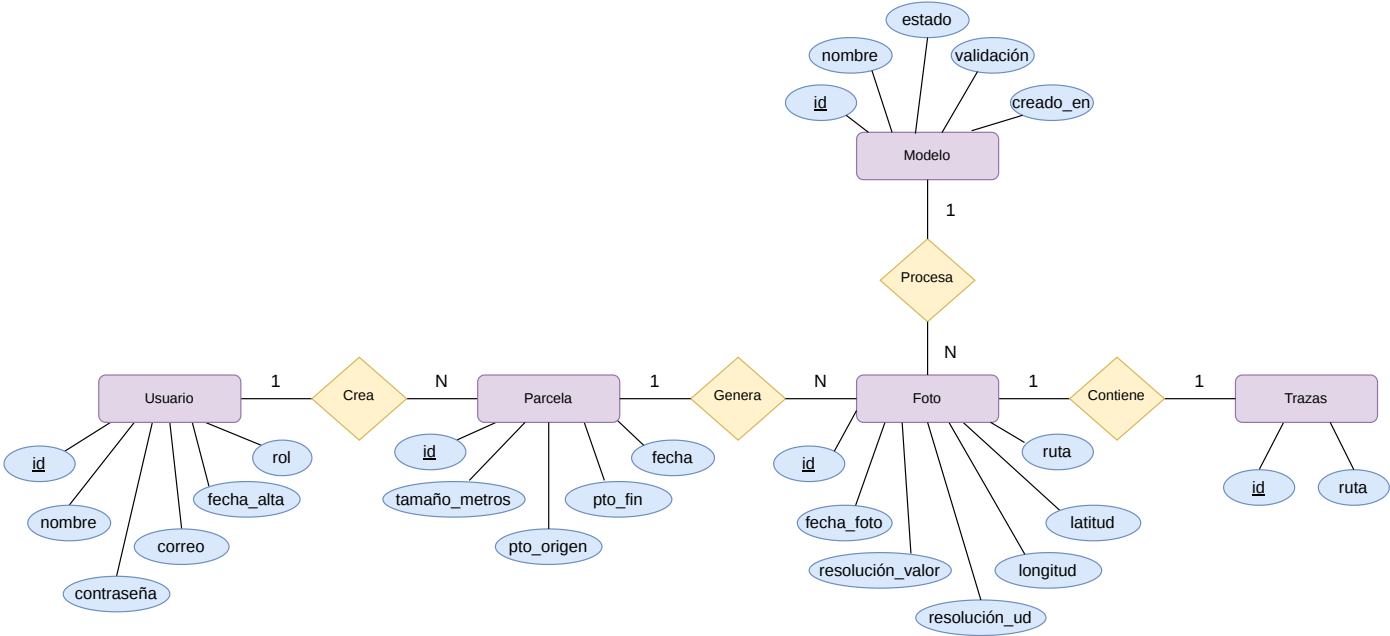


Figura C.1: Diagrama entidad-relación de la base de datos.

Modelo Entidad–Relación

El diagrama general E/R se muestra en la Figura C.1. Para el manejo de la información necesaria en el flujo de la aplicación se han definido las siguientes entidades, junto con sus relaciones y cardinalidades:

- **Usuario:** representa a la persona registrada en la aplicación y constituye el origen del control de acceso. Incluye un identificador que actúa como clave primaria, así como atributos de autenticación y perfil, como el nombre de usuario, el correo electrónico, la contraseña, el rol y la fecha de alta. Un usuario puede crear múltiples parcelas, lo que queda reflejado mediante una relación uno a muchos.
- **Parcela:** representa una unidad de trabajo asociada a un usuario, entendida como el área seleccionada por el mismo sobre la que se pretende organizar la información de análisis. Dispone de un identificador como clave primaria y, además, almacena atributos espaciales y descriptivos, como el tamaño en metros, el punto de origen, el punto final y una fecha asociada a la parcela. Cada una de ellas puede generar un *grid* con múltiples fotos, estableciendo de nuevo una relación uno a muchos.
- **Modelo:** representa los modelos de segmentación disponibles en el sistema y permite mantener un registro de los recursos de inferencia que pueden utilizarse para generar trazas. Cada entidad se corresponde con un modelo físico almacenado en la carpeta de modelos de la aplicación. Incluye un identificador como clave primaria y atributos como el nombre del modelo, su estado, su validación y la fecha de creación. Esta entidad permite conocer qué modelos existen, cuál se encuentra activo y cuáles han sido validados antes de utilizarse en el proceso de segmentación. Además, un mismo modelo puede procesar múltiples fotos, mientras que cada foto queda asociada al modelo concreto que generó sus trazas, lo que se formaliza mediante una relación uno a muchos.
- **Foto:** modela cada imagen procesada dentro de una parcela y concentra los metadatos necesarios para su trazabilidad. Incluye un identificador como clave primaria y atributos como fecha de captura, resolución expresada en valor y unidad, coordenadas geográficas y rutas de almacenamiento, diferenciando la ruta de la imagen original y la ruta del resultado de trazas. Asimismo, cada foto queda vinculada al modelo utilizado durante su procesamiento, permitiendo conocer con precisión

qué configuración de inferencia produjo el resultado asociado. Una foto puede contener exactamente un conjunto de trazas, formalizado mediante una relación uno a uno.

- **Trazas:** representa el resultado generado por el sistema a partir de una foto concreta. Contiene un identificador como clave primaria y una ruta asociada al recurso resultante. Cada uno de estos ficheros `.json` está asociado a una imagen, por lo que se establece una relación uno a uno entre la foto procesada y sus trazas correspondientes.

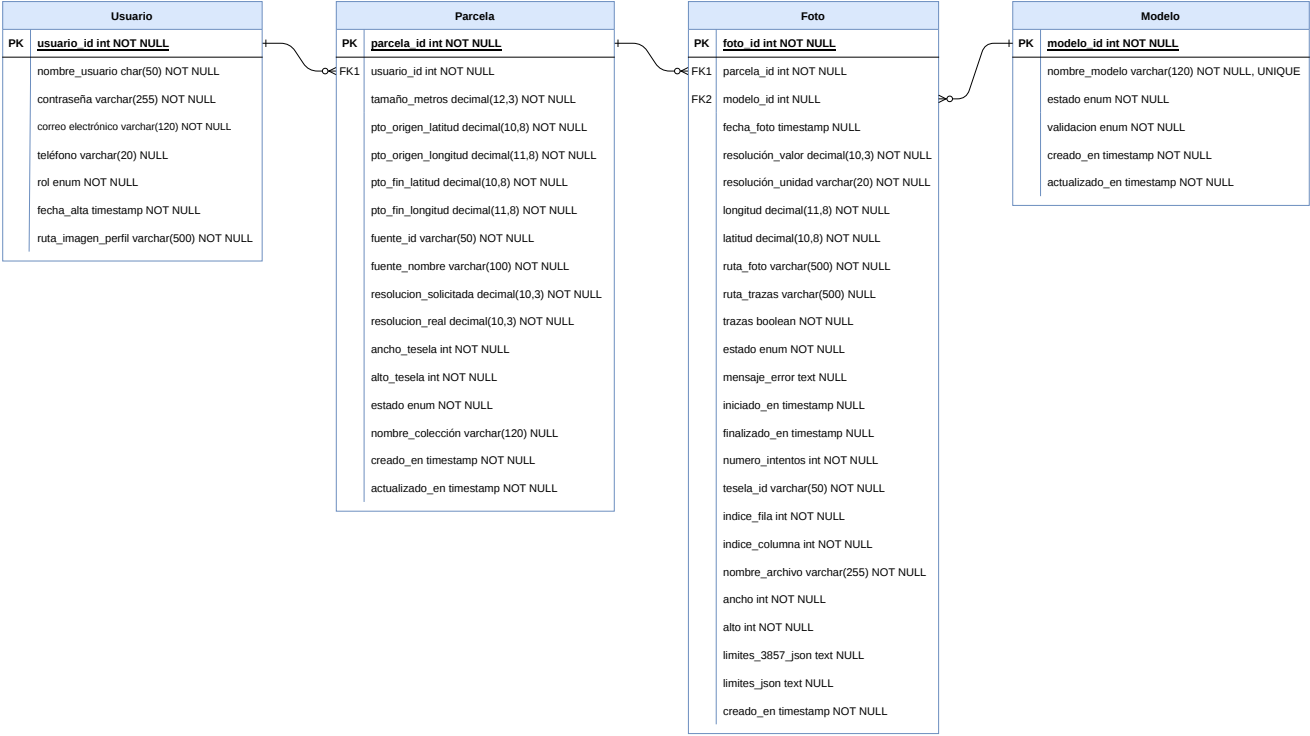


Figura C.2: Diagrama relacional de la base de datos.

Modelo Relacional

En la Figura C.2 se recoge la traducción del modelo conceptual al esquema relacional, estructurado en las tablas *Usuario*, *Parcela*, *Foto* y *Modelo*. La tabla *Usuario* concentra los atributos de identificación, autenticación y perfil, incluyendo el nombre de usuario, la contraseña cifrada, el correo electrónico, el rol, la fecha de alta, el teléfono y la ruta de la imagen de perfil. A partir de ella, *Parcela* actúa como entidad dependiente mediante la clave foránea `usuario_id`, estableciendo una relación uno a muchos que permite organizar el histórico de zonas de análisis asociadas a cada usuario.

La tabla *Parcela* incorpora la información necesaria para describir cada área seleccionada, incluyendo el tamaño en metros, las coordenadas de origen y fin, la fuente de datos utilizada, la resolución solicitada y real, las dimensiones de tesela, el estado de procesamiento, el nombre de la colección y las marcas temporales de creación y actualización. De este modo, cada parcela funciona como contenedor lógico de un conjunto de imágenes derivadas del recorte geográfico definido por el usuario. A su vez, la tabla *Foto* se vincula a *Parcela* mediante `parcela_id`, almacenando los metadatos específicos de cada tesela o imagen, como resolución, latitud, longitud, ruta del fichero original, identificador de tesela, posición dentro del *grid*, dimensiones, límites geográficos y estado de ejecución.

En este esquema relacional, las trazas no se modelan como una tabla independiente, sino como un resultado asociado directamente a cada registro de *Foto*. Para ello, se emplean atributos como `ruta_trazas`, que almacena la ruta al fichero generado tras la segmentación, y `trazas`, que indica si existe un resultado calculado para esa imagen. Además, se incorporan campos de control como `estado`, `mensaje_error`, `iniciado_en`, `finalizado_en` y `numero_intentos`, lo que permite representar el ciclo de vida del procesamiento y gestionar fallos o reintentos de forma trazable.

Por último, el esquema cuenta con la tabla *Modelo*, que registra los modelos de segmentación disponibles en el sistema. Cada modelo queda identificado mediante `modelo_id` y `nombre_modelo`, y se caracteriza mediante atributos como `estado`, `validacion`, `creado_en` y `actualizado_en`. La relación con *Foto* se lleva a cabo mediante la clave foránea `modelo_id`, de forma que un mismo modelo puede haber procesado múltiples imágenes, mientras que cada foto conserva la referencia al modelo concreto que generó sus trazas. Esto permite conocer, no solo si una imagen dispone de trazas calculadas, sino también con qué modelo fueron obtenidas.

Diccionario de datos

A continuación se presenta el diccionario de datos de las tablas principales del sistema. En cada tabla se recoge el nombre del atributo, el tipo de dato previsto, su papel dentro del esquema relacional y una breve descripción de su función.

Nombre	Tipo	Columna	Descripción
usuario_id	INTEGER	PK	Identificador único del usuario.
nombre_usuario	CHAR(50)	NOT NULL	Nombre de usuario empleado para identificar la cuenta dentro de la aplicación.
contrasena	VARCHAR(255)	NOT NULL	Hash de la contraseña del usuario.
correo_ electronico	VARCHAR(120)	UNIQUE	Correo electrónico del usuario, utilizado como dato identificativo único.
telefono	VARCHAR(20)	NULL	Teléfono opcional del usuario, almacenado de forma normalizada.
rol	ENUM	NOT NULL	Rol funcional del usuario, empleado para determinar sus permisos de acceso.
fecha_alta	TIMESTAMP	NOT NULL	Fecha y hora de creación de la cuenta.
ruta_imagen_ perfil	VARCHAR(500)	NULL	Ruta relativa de la imagen de perfil del usuario.

Tabla C.1: Diccionario de datos correspondiente a la tabla *Usuario*.

Nombre	Tipo	Columna	Descripción
parcela_id	INTEGER	PK	Identificador único de la parcela.
usuario_id	INTEGER	FK	Usuario propietario de la parcela.
tamano_metros	DECIMAL(12,3)	NOT NULL	Tamaño aproximado de la zona seleccionada, expresado en metros.
pto_origen_latitud	DECIMAL(10,8)	NOT NULL	Latitud del punto inicial seleccionado en el visor.
pto_origen_longitud	DECIMAL(11,8)	NOT NULL	Longitud del punto inicial seleccionado en el visor.
pto_fin_latitud	DECIMAL(10,8)	NOT NULL	Latitud del punto final seleccionado en el visor.
pto_fin_longitud	DECIMAL(11,8)	NOT NULL	Longitud del punto final seleccionado en el visor.
fuelle_id	VARCHAR(50)	NOT NULL	Identificador técnico de la fuente cartográfica utilizada.
fuelle_nombre	VARCHAR(100)	NOT NULL	Nombre legible de la fuente cartográfica.
resolucion_solicitada	DECIMAL(10,3)	NOT NULL	Resolución inicialmente solicitada para la descarga o visualización.
resolucion_real	DECIMAL(10,3)	NOT NULL	Resolución finalmente utilizada por el sistema.
ancho_tesela	INTEGER	NOT NULL	Anchura en píxeles de cada tesela generada.
alto_tesela	INTEGER	NOT NULL	Altura en píxeles de cada tesela generada.
estado	ENUM	NOT NULL	Estado global de procesamiento de la parcela.
nombre_coleccion	VARCHAR(120)	NULL	Nombre visible asignado por el usuario a la colección.
creado_en	TIMESTAMP	NOT NULL	Fecha y hora de creación de la parcela.

Continúa en la página siguiente

Nombre	Tipo	Columna	Descripción
actualizado_en	TIMESTAMP	NOT NULL	Fecha y hora de la última modificación de la parcela.

Tabla C.2: Diccionario de datos correspondiente a la tabla *Parcela*.

Nombre	Tipo	Columna	Descripción
foto_id	INTEGER	PK	Identificador único de la foto.
parcela_id	INTEGER	FK	Parcela a la que pertenece la foto.
modelo_id	INTEGER	FK, NULL	Modelo de segmentación utilizado para generar las trazas. Puede ser nulo si la foto todavía no se ha procesado.
fecha_foto	TIMESTAMP	NULL	Fecha asociada a la imagen.
resolucion_valor	DECIMAL(10,3)	NOT NULL	Valor numérico de la resolución espacial.
resolucion_unidad	VARCHAR(20)	NOT NULL	Unidad de resolución, como por ejemplo m/px.
longitud	DECIMAL(11,8)	NOT NULL	Longitud geográfica asociada a la tesela.
latitud	DECIMAL(10,8)	NOT NULL	Latitud geográfica asociada a la tesela.
ruta_foto	VARCHAR(500)	NOT NULL	Ruta relativa al archivo físico de imagen.
ruta_trazas	VARCHAR(500)	NULL	Ruta relativa al archivo físico de trazas generado.
trazas	BOOLEAN	NOT NULL	Indica si la foto tiene trazas generadas.
estado	ENUM	NOT NULL	Estado de procesamiento de la foto.
mensaje_error	TEXT	NULL	Detalle del error en caso de fallo de procesamiento.

Continúa en la página siguiente

Nombre	Tipo	Columna	Descripción
iniciado_en	TIMESTAMP	NULL	Fecha y hora de inicio del procesamiento.
finalizado_en	TIMESTAMP	NULL	Fecha y hora de finalización del procesamiento.
numero_intentos	INTEGER	NOT NULL	Número de intentos de procesamiento realizados.
tesela_id	VARCHAR(50)	NOT NULL	Identificador interno de la tesela dentro de la cuadrícula.
indice_fila	INTEGER	NOT NULL	Fila de la tesela dentro de la cuadrícula.
indice_columna	INTEGER	NOT NULL	Columna de la tesela dentro de la cuadrícula.
nombre_archivo	VARCHAR(255)	NOT NULL	Nombre del archivo físico de la tesela.
ancho	INTEGER	NOT NULL	Anchura en píxeles de la imagen.
alto	INTEGER	NOT NULL	Altura en píxeles de la imagen.
limites_3857_json	TEXT	NULL	Límites de la tesela en EPSG:3857 almacenados como JSON serializado.
limites_json	TEXT	NULL	Límites geográficos de la tesela almacenados como JSON serializado.
creado_en	TIMESTAMP	NOT NULL	Fecha y hora de creación del registro.

Tabla C.3: Diccionario de datos correspondiente a la tabla *Foto*.

Nombre	Tipo	Columna	Descripción
modelo_id	INTEGER	PK	Identificador único del modelo de segmentación.
nombre_modelo	VARCHAR(120)	NOT NULL, UNIQUE	Nombre del modelo dentro del sistema, normalmente coincidente con el archivo físico del mismo.

Continúa en la página siguiente

Nombre	Tipo	Columna	Descripción
estado	ENUM	NOT NULL	Indica si el modelo está activo o no. El modelo activo se utiliza para nuevas inferencias.
validacion	ENUM	NOT NULL	Indica si el modelo ha superado la validación previa a su uso.
creado_en	TIMESTAMP	NOT NULL	Fecha y hora en la que el modelo fue registrado.
actualizado_en	TIMESTAMP	NOT NULL	Fecha y hora de la última actualización del modelo.

Tabla C.4: Diccionario de datos correspondiente a la tabla *Modelo*.

C.3. Diseño procedimental

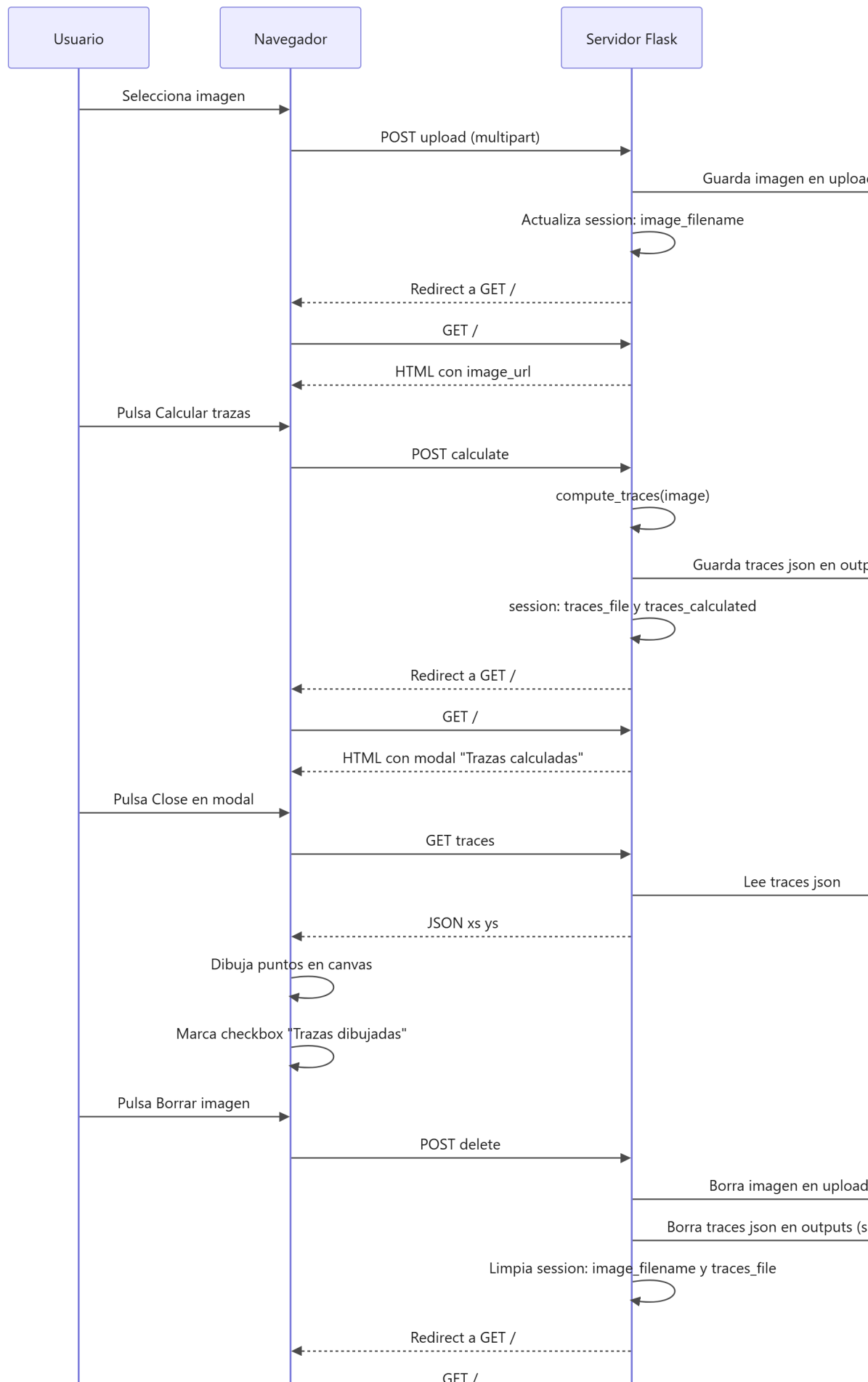
En esta sección se describe el comportamiento dinámico de la aplicación a través de los principales flujos de interacción entre el usuario y el sistema. Para ello, se emplean diagramas de secuencia que permiten representar, de forma ordenada, las peticiones realizadas desde el navegador, la respuesta del servidor Flask, el acceso a la persistencia y la generación o visualización de resultados.

La Figura C.3 describe de forma cronológica el flujo de interacción entre el usuario, el navegador, el servidor Flask y el sistema de almacenamiento, permitiendo comprender el comportamiento dinámico de la aplicación.

El proceso se inicia cuando el usuario selecciona una imagen desde la interfaz. El navegador envía dicha imagen al servidor mediante una petición *POST upload*, tras lo cual Flask la persiste en disco y actualiza el estado de la sesión. A continuación, el servidor responde con una redirección que provoca la recarga de la página principal, mostrando la imagen cargada.

Posteriormente, cuando el usuario solicita el cálculo de trazas, el servidor ejecuta la lógica correspondiente, genera los resultados en formato JSON y los almacena en el sistema de ficheros, actualizando de nuevo la sesión para reflejar el nuevo estado. La respuesta se materializa en una recarga de la interfaz que incluye un modal de confirmación.

Una vez cerrado el modal, el navegador solicita explícitamente los resultados mediante una petición *GET traces*. El servidor lee el fichero JSON



previamente generado y lo devuelve al cliente, donde el código JavaScript se encarga de dibujar las trazas sobre el *canvas* y reflejar visualmente el estado de la aplicación.

Finalmente, si el usuario decide eliminar la imagen, se envía una petición del tipo *post* a la ruta *delete* del sistema. El servidor elimina tanto la imagen como los resultados asociados, limpia la información de la sesión y devuelve la aplicación a su estado inicial mediante una nueva recarga de la página.

C.4. Diseño arquitectónico

En esta sección se describe la arquitectura general de la aplicación, presentando la organización de los principales módulos software y de las relaciones que se establecen entre ellos. El sistema desarrollado corresponde a una aplicación web monolítica modular, implementada con Flask y Python.

Con el fin de representar esta estructura desde distintos niveles de abstracción, se incluyen cuatro diagramas complementarios: una vista conceptual de tipo hexagonal, una vista por capas, un diagrama de componentes y un diagrama de despliegue. Cada uno de ellos permite analizar la arquitectura desde una perspectiva diferente, facilitando la comprensión tanto de la organización lógica del sistema como de su comportamiento en ejecución.

Vista conceptual de la arquitectura

La Figura C.4 muestra una interpretación conceptual de la arquitectura mediante un esquema hexagonal [4]. Esta vista no pretende representar una implementación estricta basada en puertos e interfaces formales, sino explicar la separación lógica entre el dominio del problema, los casos de uso de la aplicación y los adaptadores técnicos que permiten ejecutar dichos casos sobre tecnologías concretas.

En el centro se sitúan las entidades principales del sistema, como los usuarios, las imágenes, las parcelas, los modelos, las trazas y los estados de procesamiento. Alrededor de este núcleo se encuentran las operaciones que el sistema permite realizar, entre ellas el procesamiento de imágenes, el acceso al visor cartográfico, la gestión de cuentas, la administración de usuarios y la administración de modelos. Finalmente, la parte externa agrupa la infraestructura que conecta la aplicación con el navegador, la base de datos, el sistema de ficheros, los servicios cartográficos y el motor de inferencia.

Las flechas del diagrama representan el sentido principal de comunicación entre la aplicación y sus dependencias externas. Algunas conexiones son

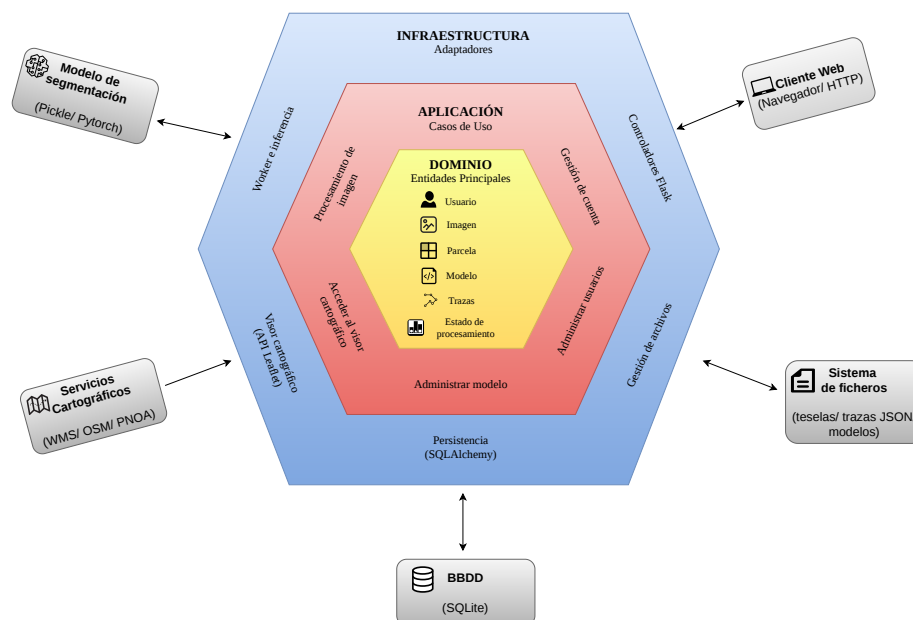


Figura C.4: Diagrama hexagonal de la arquitectura del sistema.

bidireccionales, como las establecidas con el cliente web, la base de datos y el sistema de ficheros, ya que la aplicación recibe peticiones o datos desde estos elementos y, a su vez, devuelve respuestas, consulta información o persiste nuevos resultados. En cambio, otras flechas reflejan una dependencia orientada principalmente desde el sistema hacia el exterior, como ocurre con los servicios cartográficos o con el modelo de segmentación, puesto que la aplicación los invoca para obtener imágenes, ejecutar la inferencia o generar resultados, pero el control del flujo permanece en la propia aplicación.

Arquitectura por capas

En la Figura C.5 se representa la aplicación desde una perspectiva de capas. Esta vista permite observar la separación entre la capa de presentación, la capa de negocio, la capa de datos y los servicios externos. La capa de presentación se corresponde con el cliente web, compuesto por las páginas renderizadas desde Flask, las plantillas Jinja2, el código JavaScript y los elementos interactivos de la interfaz, como el visor cartográfico o la visualización de trazas.

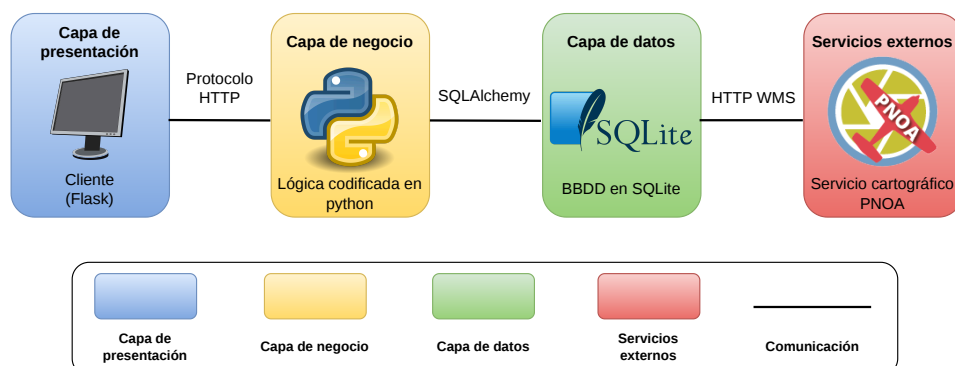


Figura C.5: Arquitectura por capas del sistema.

La capa de negocio está formada por la lógica implementada en Python dentro de la aplicación Flask. En ella se encuentran las rutas, los servicios internos, la gestión de sesiones, la autenticación, la administración, la generación de colecciones, el control de estados y la coordinación del proceso de inferencia. Esta capa actúa como núcleo operativo del sistema, recibiendo peticiones desde el navegador y transformándolas en operaciones sobre datos, ficheros, modelos o servicios externos.

La capa de datos combina dos mecanismos de persistencia. La información estructurada se almacena en SQLite mediante SQLAlchemy, incluyendo usuarios, parcelas, fotos, modelos y estados. En cambio, los recursos de mayor tamaño se conservan en el sistema de ficheros, como las imágenes subidas, las teselas generadas, los resultados en formato JSON y los modelos entrenados. Por último, los servicios externos proporcionan la información cartográfica necesaria para el flujo basado en visor, principalmente a través de peticiones WMS sobre ortofotografía del PNOA.

Diagrama de componentes

La Figura C.6 resume los componentes principales que intervienen en la ejecución de la aplicación. En esta vista se abstrae la aplicación web como el componente central, encargado de coordinar la comunicación entre el usuario, la base de datos, el sistema de ficheros y los servicios cartográficos externos. Este diagrama permite identificar qué elementos participan directamente

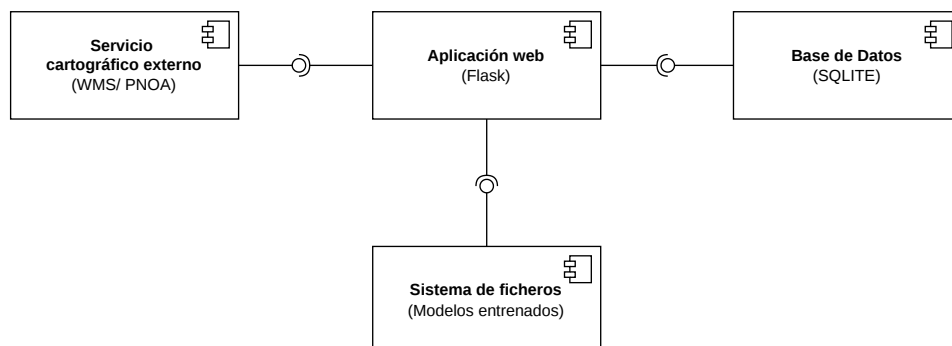


Figura C.6: Diagrama de componentes de la aplicación.

en el funcionamiento del sistema, sin entrar en el detalle interno de cada módulo.

El componente principal es la aplicación Flask, que recibe las peticiones procedentes del navegador y decide qué operación debe ejecutarse. Cuando se trabaja con imágenes individuales, el servidor valida la subida, almacena la imagen, ejecuta la inferencia y deja disponibles las trazas para su visualización. Cuando se trabaja con el visor cartográfico, el servidor registra la zona seleccionada, genera las teselas asociadas y delega el procesamiento en segundo plano al *worker*. En ambos casos, Flask actúa como punto de coordinación entre la interfaz, la persistencia y los modelos.

La base de datos almacena el estado lógico de la aplicación, mientras que el sistema de ficheros conserva los artefactos generados o utilizados durante el procesamiento. El servicio cartográfico externo proporciona las imágenes necesarias para el flujo de colecciones, y el módulo de inferencia emplea los modelos disponibles para generar las máscaras y trazas correspondientes.

Diagrama de despliegue

El diagrama de despliegue mostrado en la Figura C.7 representa una vista simplificada y abstraída de la distribución física del sistema, con el objetivo de mostrar los principales nodos de ejecución y las comunicaciones entre ellos sin entrar en detalles específicos.

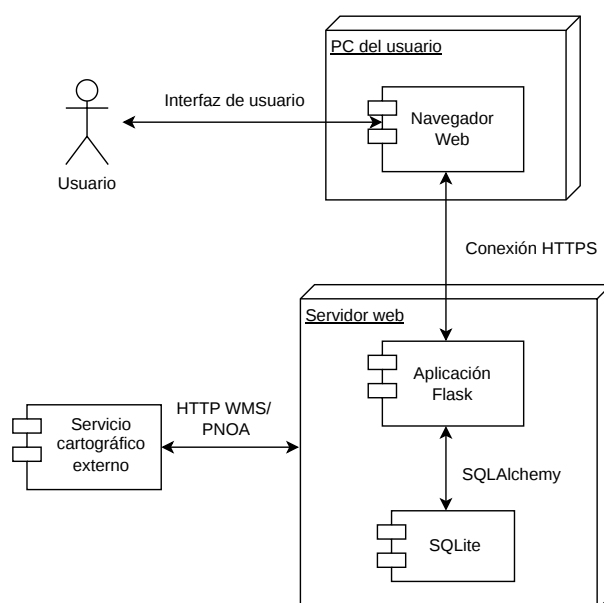


Figura C.7: Diagrama de despliegue de la aplicación.

Resumen de la arquitectura adoptada

En conjunto, la arquitectura del sistema puede describirse como un monolito Flask modular con separación pragmática de responsabilidades. No se trata de una arquitectura de microservicios, ya que todos los módulos principales se ejecutan dentro de una misma aplicación, pero tampoco es una estructura monolítica desorganizada. La aplicación distingue de forma clara entre presentación, lógica de negocio, persistencia, procesamiento de imágenes, administración y comunicación con servicios externos.

C.5. Diseño web

En este apartado se presentan los *mockups* de las principales pantallas de la aplicación, con el objetivo de concretar el flujo de interacción previsto y servir de soporte a la especificación funcional. Estas maquetas permiten validar, de forma temprana, tanto la distribución de la interfaz como la secuencia de acciones que guían al usuario a través del *pipeline* de procesamiento, desde la selección de una ortoimagen hasta la obtención y visualización de las trazas detectadas.

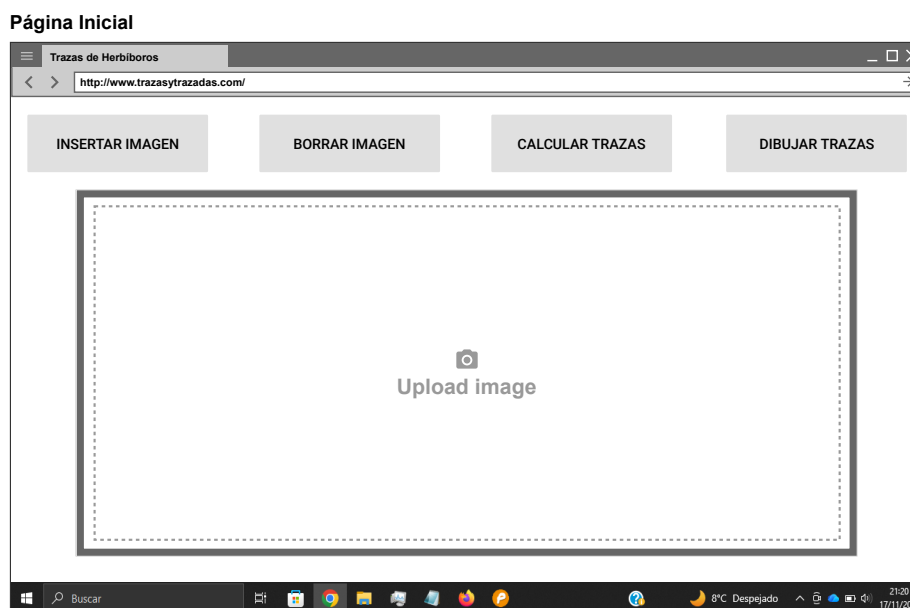


Figura C.8: Mockup de la web: página principal.

Página principal

La Figura C.8 representa el estado inicial de la aplicación, cuando todavía no se ha cargado ninguna imagen. En la parte superior se dispone una barra de acciones que agrupa las operaciones principales del sistema, insertar y borrar imagen, calcular trazas y dibujar trazas, mientras que la zona central actúa como área de trabajo, ofreciendo un contenedor de subida y previsualización. Este diseño prioriza la claridad del flujo, de modo que el usuario identifique desde el primer momento cuál es el punto de entrada del proceso y qué pasos debe ejecutar para pasar de la imagen de entrada a la representación visual del resultado.

Imagen insertada

El estado inmediatamente posterior a la página principal C.8 se alcanza cuando el usuario pulsa *Insertar imagen* y selecciona el fichero que desea analizar. En ese momento, la interfaz pasa a mostrar la imagen cargada, tal y como se aprecia en la Figura C.9. De este modo, el contenedor que en la pantalla inicial presentaba el texto *upload image* se sustituye por la previsualización de la ortoimagen, quedando esta disponible para su inspección en el cliente y para la ejecución posterior del cálculo de trazas.

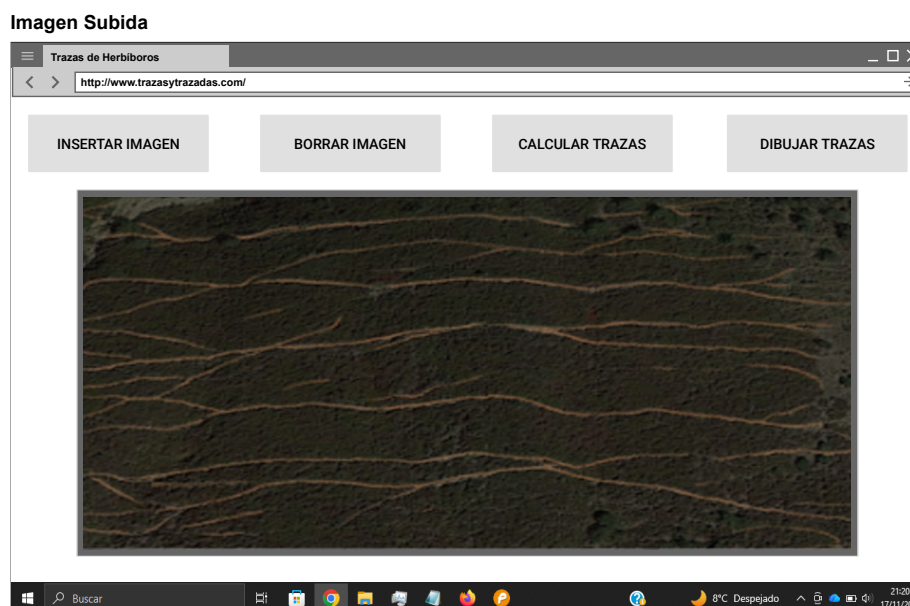


Figura C.9: Mockup de la web: imagen insertada.

Modal de trazas calculadas

Una vez se pulse el botón *Calcular trazas*, aparecerá un modal superpuesto sobre la imagen indicando que el proceso de inferencia ha finalizado y que el siguiente paso consiste en representar el resultado. Para ello, tal y como se muestra en la Figura C.10, el usuario deberá pulsar el botón *Dibujar trazas*.

Trazas dibujadas

Tras hacer uso del botón *Dibujar trazas* y, suponiendo que se ha seguido correctamente el flujo previo, se mostrará la ortoimagen original con las trazas superpuestas. De este modo, las sendas detectadas quedan resaltadas y son fácilmente inspeccionables por el usuario, tal y como se aprecia en la Figura C.11.

Modal de error

Si se interrumpe el flujo de la aplicación o se produce algún error durante la ejecución, el sistema mostrará un modal informativo indicando que no ha sido posible completar correctamente el proceso. Este comportamiento permite comunicar la incidencia de forma clara y mantener la interfaz en un estado consistente, tal y como se observa en la Figura C.12.

Calculando trazas...

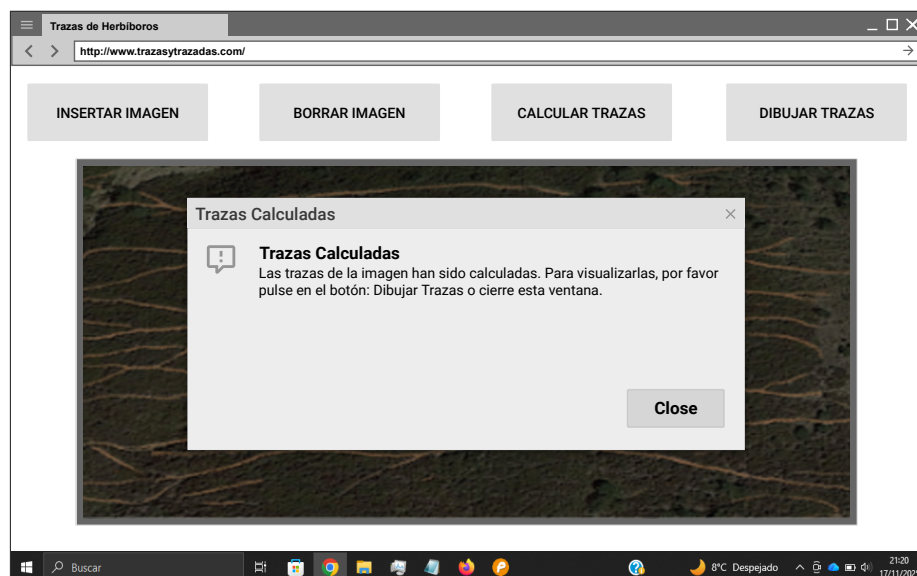


Figura C.10: Mockup de la web: modal de trazas.

Trazas Dibujadas

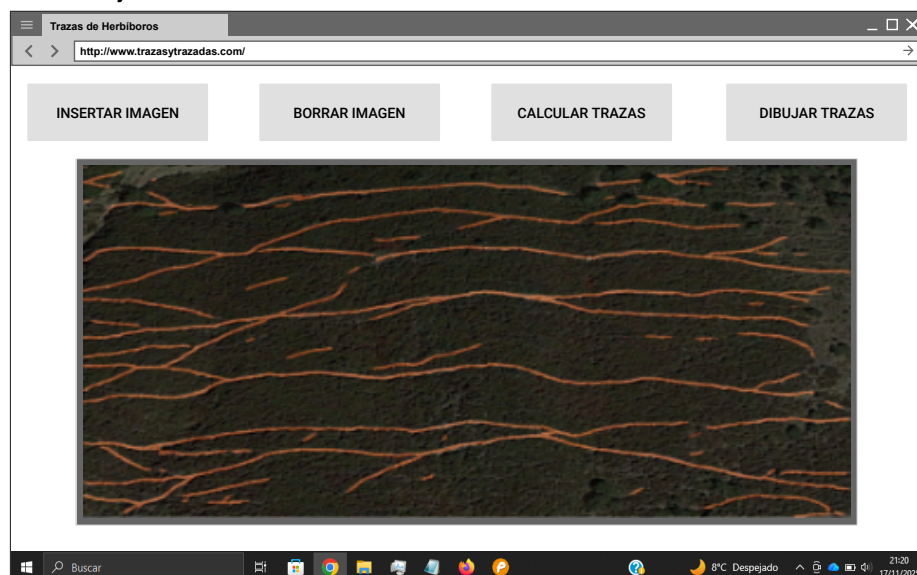


Figura C.11: Mockup de la web: trazas dibujadas.

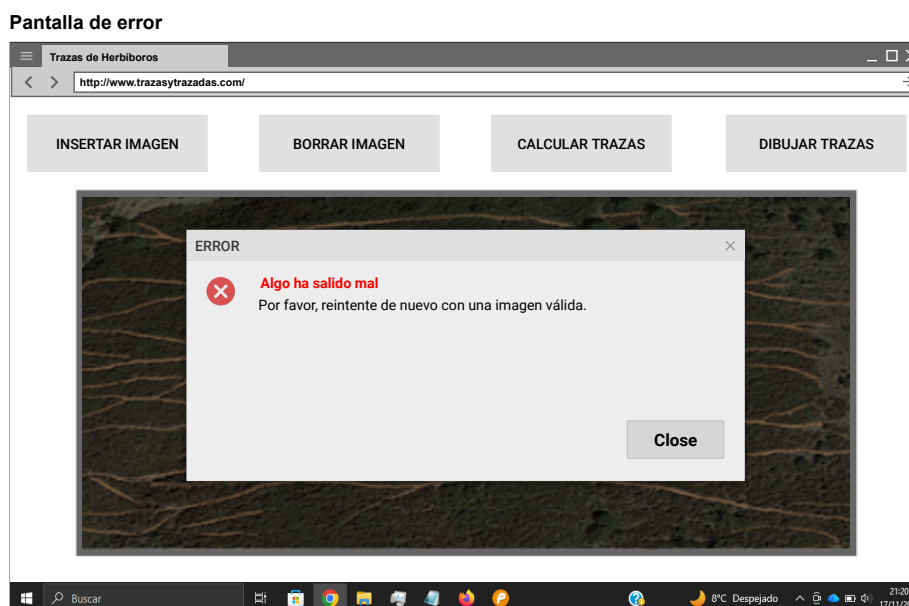


Figura C.12: Mockup de la web: modal de error.

Menú de la web

En este segundo *mockup* se aborda la ampliación funcional orientada al tratamiento geoespacial de la aplicación, incorporando un flujo de trabajo basado en selección de zonas, generación de teselas y gestión de resultados.

En primer lugar, se introduce un menú de navegación, tal y como se observa en la Figura C.13. Esta incorporación resulta necesaria al evolucionar la aplicación desde una única pantalla hacia un conjunto de vistas diferenciadas, permitiendo el acceso rápido a cada módulo y garantizando una navegación clara entre funcionalidades.

Visor

La siguiente pantalla corresponde al visor cartográfico, mostrado en la Figura C.14. En esta vista se integrará un mapa interactivo mediante *Leaflet*, capaz de cargar ortofotografía del PNOA a niveles de *zoom* adecuados. Una vez alcanzada la escala de trabajo, el usuario seleccionará dos puntos para definir un rectángulo delimitador, entendido como zona de interés. A partir de dicha geometría, el sistema generará un conjunto de teselas correspondientes al recorte definido, asociando cada imagen resultante a la zona seleccionada.



Figura C.13: Mockup de la web: menú.



Figura C.14: Mockup de la web: visor.

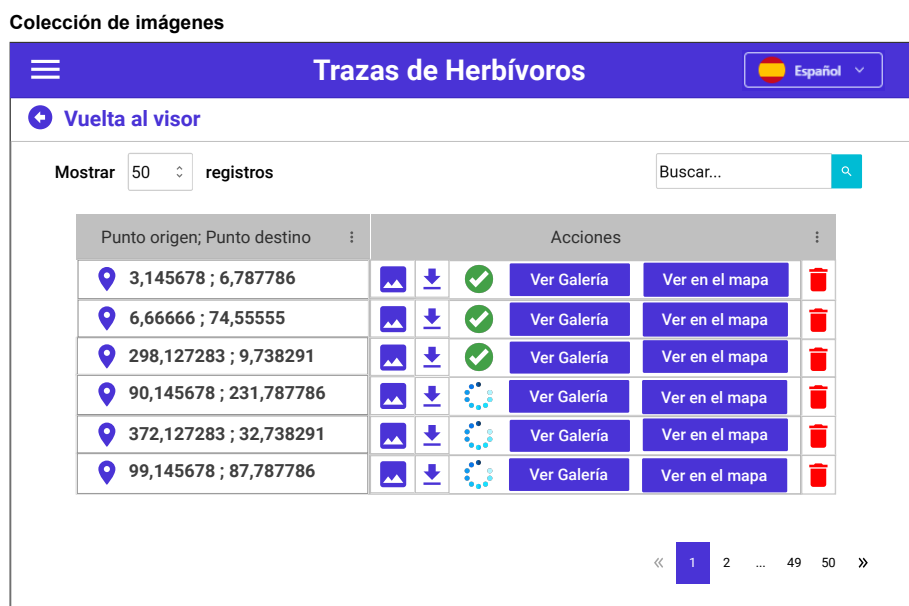


Figura C.15: Mockup de la web: colección de imágenes.

Colección de imágenes

Una vez generadas las teselas, el usuario podrá acceder a la colección de zonas registradas, representada en la Figura C.15. Esta pantalla organiza en una tabla todas las zonas creadas, mostrando como metadatos informativos las coordenadas de origen y fin del recuadro, junto con un conjunto de acciones asociadas. Entre ellas, se contempla la previsualización de la zona, la descarga de un fichero `.zip` con las teselas generadas y un indicador de estado que refleje si el cálculo de trazas ya se ha completado para todas las imágenes. Adicionalmente, se incluyen accesos directos a la galería, a la visualización de la zona en el mapa, y una opción de eliminación permanente del registro, junto con un control de paginación para navegar entre entradas.

Galería de imágenes

Al acceder a la galería, se presenta un listado de las imágenes asociadas a la zona seleccionada, tal y como se observa en la Figura C.16. Cada elemento incluye su identificación y metadatos básicos, además de un indicador de progreso que señala si las trazas han sido calculadas. Este cálculo se plantea como un proceso automático, de modo que el usuario pueda supervisar el estado sin intervención adicional.



Figura C.16: Mockup de la web: galería de imágenes.

Imagen resaltada

Por último, la Figura C.17 muestra la vista de detalle de una imagen individual. En esta pantalla se habilita la consulta de la imagen a mayor resolución y la inspección de sus características, incorporando además acciones explícitas sobre el procesado. En particular, se incluye un botón para dibujar las trazas, únicamente cuando estas hayan sido calculadas previamente, y un botón para recalcularlas en caso de fallo o inconsistencia durante la ejecución automática.

Registro

En este tercer *mockup* se aborda la incorporación de la lógica de usuarios en la aplicación, introduciendo un flujo de autenticación con registro e inicio de sesión, una gestión básica del perfil y un conjunto de vistas restringidas a usuarios con rol de administrador. Esta ampliación permite habilitar funcionalidades que requieren persistencia y control de permisos, manteniendo a su vez un modo de uso público con capacidades limitadas.

La Figura C.18 presenta la pantalla de registro, concebida como punto de entrada para la creación de una cuenta. La interfaz organiza el formulario en dos columnas para separar datos de identificación y contacto, como

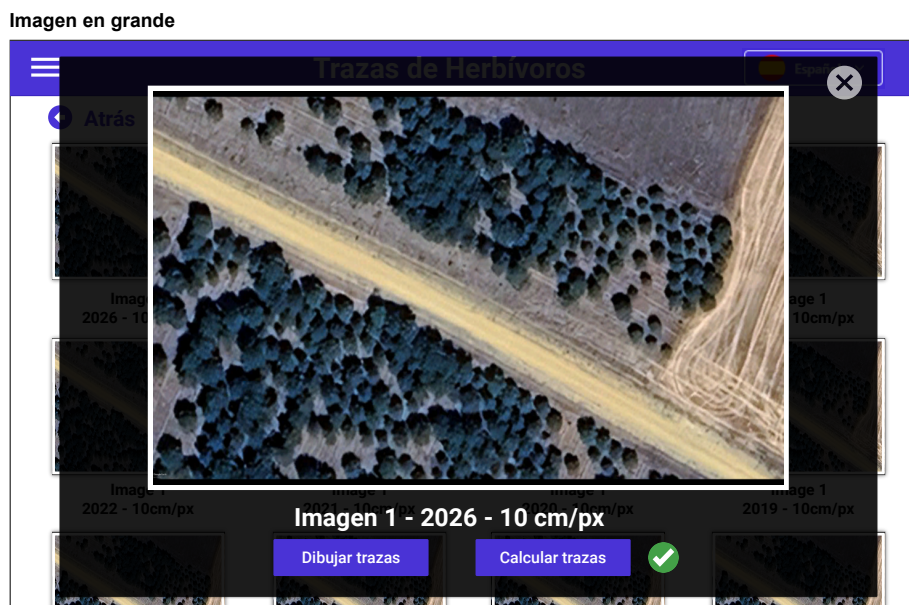


Figura C.17: Mockup de la web: imagen resaltada.

Registro

The mockup shows a registration form titled 'REGISTRARSE' in blue. The form is divided into two columns. The left column contains three input fields for 'Usuario' (with example 'Ej: Pepe1234'), 'Correo Electrónico' (with example 'Ej: pepe1234@gmail.com'), and 'Teléfono (opcional)' (with example 'Ej: +34 999 99 99 99'). The right column contains two input fields for 'Contraseña' (with placeholder '(Introduzca aquí la contraseña)') and 'Repetir Contraseña' (with placeholder '(Repita la contraseña)'). Below these fields is a blue 'Registrarse' button. At the bottom right, there is a link that says '¿Ya tiene cuenta? Iniciar Sesión'.

Figura C.18: Mockup de la web: registro.

Inicio De Sesión

The mockup shows a login interface with a light green background featuring a blurred image of a forest. The interface is divided into two main sections by a vertical line. The left section, titled 'BIENVENIDO A LA APP DE DETECCIÓN DE TRAZAS', contains three paragraphs of text explaining the app's features and usage. The right section, titled 'INICIO DE SESIÓN', contains a login form with fields for 'Usuario' and 'Contraseña', each with a placeholder text '(Introduzca aquí su usuario)' and '(Introduzca aquí su contraseña)' respectively. Below these fields is a blue button labeled 'Iniciar Sesión' and a link labeled '¿Registrarse?'.

BIENVENIDO A LA APP DE DETECCIÓN DE TRAZAS

Aquí podrás analizar imágenes para detectar y visualizar trazas asociadas a la presión herbívora.

Sin iniciar sesión, puedes trabajar con imágenes sueltas: cargarlas, calcular sus trazas y dibujarlas sobre la propia imagen.

Si accedes con tu cuenta, además dispondrás de herramientas cartográficas y de gestión: seleccionar zonas en el visor, generar y descargar teselas de imágenes, guardar las zonas en tu colección y consultarlas más tarde.

Para iniciar sesión, introduce tu nombre de usuario y tu contraseña.

INICIO DE SESIÓN

Usuario
(Introduzca aquí su usuario)

Contraseña
(Introduzca aquí su contraseña)

Iniciar Sesión

[¿Registrarse?](#)

Figura C.19: Mockup de la web: log in.

usuario, correo electrónico y un campo teléfono opcional, de los campos de credenciales, como contraseña y repetición de la misma, reduciendo errores de introducción.

Tras completar los campos, el usuario confirma la operación mediante el botón *Registrarse*, y se incluye un enlace alternativo para acceder directamente a la pantalla de inicio de sesión cuando ya se dispone de una cuenta.

Inicio de Sesión

La Figura C.19 muestra la pantalla de inicio de sesión, destinada a autenticar al usuario y habilitar las funcionalidades asociadas a su rol. La interfaz se divide en dos secciones: a la izquierda se presenta una descripción breve del propósito de la aplicación y una diferenciación entre el modo de uso sin autenticación, orientado al procesamiento de imágenes individuales, y el modo autenticado, que habilita herramientas cartográficas y gestión persistente de colecciones.

A la derecha se dispone el formulario de acceso, compuesto por los campos de usuario y contraseña y el botón *Iniciar Sesión*, junto con un enlace directo a la pantalla de registro para usuarios que todavía no dispongan de cuenta.



Figura C.20: Mockup de la web: Menú de usuarios.

Menú de usuarios

La Figura C.20 muestra el menú de navegación una vez el usuario ha iniciado sesión. Su estructura es análoga a la del menú introducido en el *mockup* de la parte geoespacial, manteniendo las entradas principales de la aplicación, como *Home*, *Visor* y *Colección de imágenes*. No obstante, en esta versión se incorporan las nuevas pantallas asociadas a la gestión de usuarios, incluyendo el acceso al perfil y la opción de cierre de sesión.

Adicionalmente, se añade un *panel del administrador* que agrupa funcionalidades de mantenimiento internas, como la gestión de usuarios y la gestión del modelo, el cual únicamente se muestra cuando el rol autenticado corresponde a administrador, de modo que el menú se adaptará dinámicamente a los permisos disponibles.

Gestión de usuarios

La Figura C.21 muestra la pantalla de *Gestión de usuarios*, accesible exclusivamente para el rol de administrador. Esta vista centraliza la supervisión y administración de cuentas registradas, presentando un resumen superior con métricas agregadas, como el número total de usuarios, la cantidad de

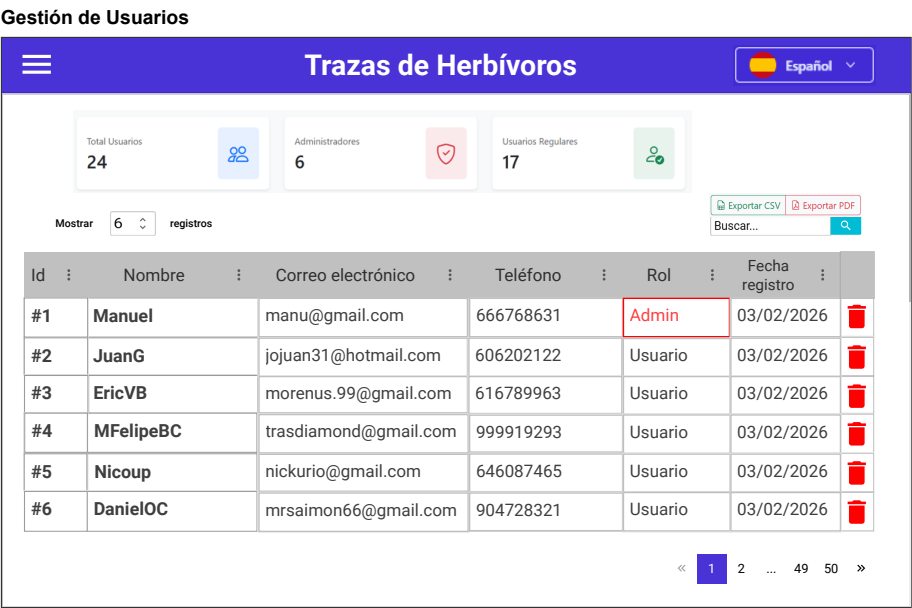


Figura C.21: Mockup de la web: Gestión de usuarios.

administradores y el número de usuarios regulares, y una tabla principal con los registros disponibles.

En dicha tabla se incluyen campos identificativos y de contacto, junto con el rol asignado y la fecha de registro, incorporando además mecanismos habituales de explotación de listados, como búsqueda, paginación y exportación de resultados. Finalmente, se contempla una acción de eliminación por usuario, destinada a operaciones de mantenimiento y control del sistema, manteniendo la coherencia con el modelo de permisos, de modo que solo el administrador pueda ejecutar modificaciones sobre el conjunto global de cuentas.

Gestión de modelos

La Figura C.22 muestra la pantalla de *Gestión de modelos*, concebida como una funcionalidad exclusiva del rol de administrador para supervisar y ajustar el modelo de inferencia empleado por el sistema. En esta vista se incluye un selector que lista los distintos archivos disponibles e indica explícitamente cuál se encuentra activo, permitiendo seleccionar un modelo alternativo para su comparación.



Figura C.22: Mockup de la web: Gestión de modelos.

A continuación, se habilita un área de carga de imagen destinada a pruebas controladas, de modo que la misma entrada pueda procesarse con el modelo activo y con el modelo seleccionado, mostrando ambos resultados en paralelo para facilitar una inspección cualitativa de diferencias de segmentación. Finalmente, se incorpora un botón para establecer el modelo seleccionado como predeterminado, actualizando la configuración del sistema de forma persistente para futuras ejecuciones.

Gestión de cuenta

La Figura C.23 muestra la pantalla de *Gestión de cuenta*, destinada a centralizar la información básica del usuario autenticado y a ofrecer accesos directos a las funcionalidades principales asociadas a su sesión. En esta vista se presentan los datos de perfil, incluyendo nombre de usuario, correo electrónico, teléfono y fecha de registro, junto con una acción para *Editar información*, orientada a la actualización de los campos permitidos.

Asimismo, se incorporan accesos rápidos al visor y a la colección de imágenes, con el objetivo de reducir fricción en la navegación y facilitar el retorno a los módulos de trabajo. Finalmente, se incluye la opción de *Cerrar*

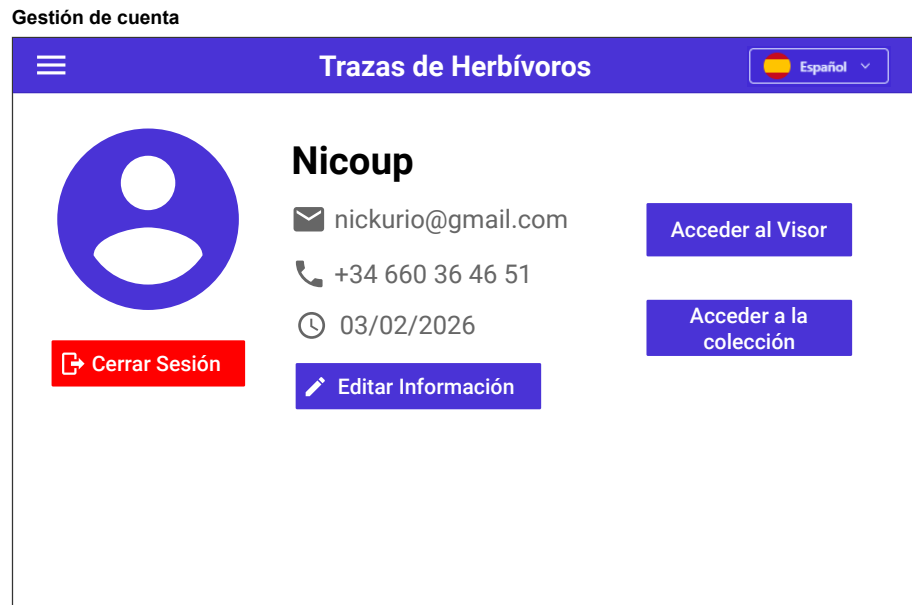


Figura C.23: Mockup de la web: Gestión de cuenta.

sesión, que invalida la sesión activa y devuelve la aplicación al modo de acceso no autenticado.

Apéndice D

Documentación técnica de programación

D.1. Introducción

En esta sección se recoge la documentación técnica asociada al desarrollo de la aplicación, con el objetivo de facilitar la comprensión interna del proyecto y servir como punto de partida para futuros desarrolladores que deban mantenerlo, ampliarlo o desplegarlo. Para ello, se describe la organización de los directorios principales, el contenido más relevante de cada uno de ellos y la función que desempeñan dentro del sistema.

Además, se incluye una guía introductoria para la puesta en marcha del entorno de desarrollo, junto con las indicaciones necesarias para instalar las dependencias, ejecutar la aplicación y comprender el flujo básico de trabajo. Finalmente, se documentan las pruebas realizadas para validar el comportamiento del sistema, prestando especial atención a aquellas funcionalidades críticas relacionadas con la carga de imágenes, la inferencia, la persistencia de resultados y la interacción entre los distintos módulos de la aplicación.

Repositorio GitHub

<https://github.com/Marcurio22/TFG-Marcos-Zamora>

D.2. Estructura de directorios

La estructura del repositorio se ha organizado con el objetivo de separar el código fuente de la aplicación, la configuración del entorno, la documentación auxiliar, los datos locales generados durante la ejecución y la batería de pruebas. Esta organización facilita que un desarrollador que se incorpore al proyecto pueda identificar rápidamente dónde se encuentra cada recurso y qué papel desempeña dentro del sistema.

Antes de describir los ficheros principales del repositorio, conviene presentar la organización general de directorios sobre la que se estructura la aplicación:

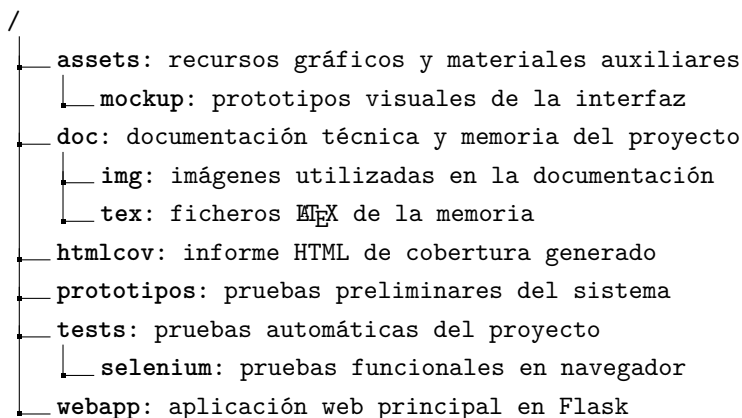


Figura D.1: Estructura principal de directorios del proyecto.

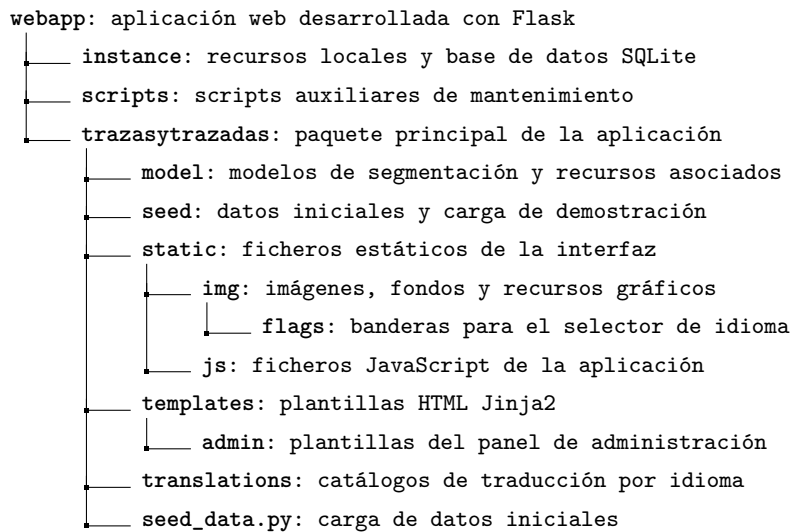


Figura D.2: Estructura principal de directorios de la aplicación web.

Los ficheros principales que se encuentran al nivel raíz del proyecto son:

- **webapp/**: directorio raíz de la aplicación web. Contiene el punto de entrada de Flask, el código fuente, los recursos estáticos, las plantillas y los ficheros propios de ejecución de la aplicación.
- **tests/**: directorio que contiene la suite de pruebas automáticas del sistema. Incluye pruebas funcionales, de regresión, de integración y pruebas de navegador con Selenium.
- **htmlcov/**: informe HTML generado por *coverage*. No forma parte del código fuente de la aplicación, sino que es un artefacto generado durante la validación.
- **pytest.ini**: fichero de configuración de *pytest*. Define las rutas de búsqueda, el *pythonpath* del proyecto y los marcadores específicos utilizados en la suite.
- **.coveragerc**: fichero de configuración de *coverage*, utilizado para medir la cobertura sobre el paquete principal de la aplicación.

Dentro de **webapp/** se encuentran los ficheros propios de la aplicación, entre los que destacan:

- **run.py**: punto de entrada de la aplicación Flask.

- **requirements.txt**: listado principal de dependencias necesarias para instalar y ejecutar la aplicación.
- **requirements-selenium.txt**: dependencias opcionales necesarias para ejecutar las pruebas Selenium.
- **babel.cfg**: configuración utilizada por Flask-Babel para localizar y extraer textos traducibles.
- **messages.pot**: plantilla base de traducciones generada a partir de los textos extraídos.
- **trazasytrazadas/**: paquete principal de código fuente de la aplicación.

D.3. Manual del programador

D.4. Compilación, instalación y ejecución del proyecto

D.5. Pruebas del sistema

Apéndice E

Documentación de usuario

- E.1. Introducción
- E.2. Requisitos de usuarios
- E.3. Instalación
- E.4. Manual del usuario

Anexo de sostenibilización curricular

F.1. Introducción

Este apartado recoge una reflexión personal sobre los aspectos de sostenibilidad trabajados durante el desarrollo del TFG. Aunque el proyecto se ha construido desde una perspectiva técnica, basada en una aplicación web, modelos de segmentación semántica, visor cartográfico y gestión de resultados, su finalidad está relacionada con la observación del medio natural y con la interpretación de patrones asociados a la presión de herbivoría. Por ello, la sostenibilidad no aparece como un bloque añadido al final, sino como una dimensión transversal que influye en la forma de entender el problema.

La CRUE, en sus directrices para la introducción de la sostenibilidad en el currículum universitario, plantea la necesidad de formar profesionales capaces de tomar decisiones incorporando criterios ambientales, sociales y éticos [1]. Desde mi punto de vista, este TFG encaja con esa idea porque intenta trasladar un proceso de análisis ambiental, inicialmente más cercano a un entorno de investigación, a una aplicación que pueda ser utilizada y estudiada.

F.2. Relación del proyecto con la sostenibilidad

El proyecto se vincula principalmente con la sostenibilidad ambiental, ya que trabaja sobre imágenes aéreas y zonas cartográficas para detectar

trazas de herbivoría. Estas trazas pueden servir como indicios visuales de actividad animal intensa y, por tanto, como apoyo para analizar dinámicas espaciales en el territorio. En este sentido, el trabajo se relaciona con el ODS 15, relativo a la vida de ecosistemas terrestres, al proponer una herramienta que puede contribuir a observar mejor determinados patrones del medio natural.

No obstante, he intentado no plantear la aplicación como una solución definitiva al problema ecológico. El sistema detecta píxeles compatibles con sendas mediante visión por computador, pero la interpretación ambiental final requiere contexto, conocimiento experto y prudencia. La aplicación aporta información visual y estructurada, pero debe entenderse como una herramienta de apoyo para el análisis, no como un sustituto del criterio técnico o científico.

F.3. Competencias de sostenibilidad trabajadas

Durante el desarrollo he trabajado especialmente la competencia de contextualización crítica del conocimiento. No bastaba con integrar un modelo de *deep learning* y representar su salida en pantalla; era necesario comprender qué significaba esa salida, qué límites tenía y qué implicaciones podía tener utilizarla sobre imágenes reales del territorio. Esto me obligó a conectar conceptos de visión por computador, ingeniería web, cartografía y conservación, evitando una visión puramente tecnológica del proyecto.

También he trabajado la competencia relacionada con la participación. Al desarrollar una interfaz web con carga de imágenes, visor cartográfico, colección, galería y descargas, el sistema transforma un pipeline complejo en una herramienta más accesible.

F.4. Decisiones de ingeniería aplicadas

La sostenibilidad también se ha reflejado en decisiones de ingeniería. El visor permite seleccionar una zona concreta en lugar de trabajar siempre con imágenes completas o descargas masivas. Después, el sistema divide el área en teselas, lo que permite organizar el procesamiento en unidades manejables y limitar el cálculo a la zona que realmente interesa. Además, el uso de un *worker* para procesar colecciones separa la interacción del usuario de las

tareas pesadas de inferencia, evitando bloquear la aplicación y facilitando su control.

Aun así, el proyecto también me ha hecho ser consciente del coste computacional de los modelos de aprendizaje profundo. Ejecutar inferencia sobre muchas imágenes no es gratis desde el punto de vista de recursos, por lo que una aplicación de este tipo debe controlar qué se procesa, cuándo se procesa y cómo se almacenan los resultados. En este caso, la modularización del backend, la persistencia estructurada de colecciones, la validación de modelos antes de activarlos y la separación entre imagen original, JSON de trazas y visualización superpuesta ayudan a mejorar la mantenibilidad, la trazabilidad y la reproducibilidad.

F.5. Conclusión

En conjunto, este TFG me ha permitido entender la sostenibilidad como un criterio de ingeniería y no únicamente como una cuestión ambiental. Diseñar software sostenible implica pensar en el impacto del problema que se aborda, pero también en la claridad del sistema, en su consumo de recursos, en su capacidad de ser mantenido y en la transparencia con la que muestra sus resultados. La aplicación desarrollada no resuelve por sí sola la gestión de la presión de herbivoría, pero sí aporta un soporte técnico para observar, visualizar y revisar información territorial de forma más accesible. Para mí, ese ha sido el principal aprendizaje: una solución informática tiene más valor cuando se integra con responsabilidad dentro de un contexto real y reconoce tanto sus posibilidades como sus limitaciones.

Bibliografía

- [1] CRUE - Conferencia de Rectores de las Universidades Españolas. Directrices para la introducción de la sostenibilidad en el currículo. https://www.crue.org/wp-content/uploads/2020/02/Directrices_Sostenibilidad_Crue2012.pdf, 2012. Documento institucional sobre sostenibilidad curricular en las universidades españolas. [Online; accedido 10-mayo-2026].
- [2] José Francisco Díez-Pastor, Francisco Javier González-Moya, Pedro Latorre-Carmona, Francisco Javier Pérez-Barbería, Ludmila Kuncheva, Antonio Canepa-Oneto, Álar Arnaiz-González, and César García-Osorio. Remote sensing colour image semantic segmentation of large herbivorous mammal trails. *International Journal of Remote Sensing*, 0(0):1–24, 2026.
- [3] Pallets Projects. Flask documentation — user’s guide, 2025. Accedido: 16-11-2025.
- [4] Eduardo Salguero. Arquitectura hexagonal. <https://medium.com/@edusalguero/arquitectura-hexagonal-59834bb44b7f>, 2020. Publicación en Medium. [Online; accedido 22-mayo-2026].